YOOBEE COLLEGE OF CREATIVE INNOVATION

Programme:
Master of Software Engineering

Course:
MSE907 - Industry-based Capstone Research Project

Assessment 3 – Final Research Project Report

The Usage of LLMs in Real Software Development: A Use Case from Practice

Replacing dual-entry room booking management at Crockers Property Group with the Park Tower Booking system

Industry host: Crockers Group Limited (Crockers Property Group), Auckland, New Zealand
Host supervisor: Nick Hofmann
Academic supervisor: Dr. Helena Bahrami

Andre Luiz Santos Benedetti - 270837915
Intake 2511
7th of November 2026

Declaration of Originality

I hereby declare that this project report, titled “The Usage of LLMs in Real Software Development: A Use Case from Practice”, is my own original work and has not been submitted previously, in whole or in part, for the award of any degree or diploma in any university or institution.

I have duly acknowledged all sources of information and assistance used in the preparation of this report.

I confirm that artificial intelligence tools were used responsibly in this work. Cursor was the coding assistant used to develop Park Tower Booking, as the report describes. I reviewed and edited all AI-assisted content and checked its claims, figures and tables against the project record. I accept full responsibility for the accuracy, originality and integrity of this report.

Student name: Andre Luiz Santos Benedetti

Student ID: 270837915

Date: 07/11/2026 Signature: 

Acknowledgement

I am deeply grateful to **Dr Helena Bahrami** for her invaluable guidance and support throughout the capstone. From the Research Development and Plan through the Work in Progress report to this final report, her feedback helped me turn a busy industry placement into a study with clear questions and honest limits. Her patience, her availability and the confidence she showed in the project made a real difference.

I also thank **Nick Hofmann**, my host supervisor, for the opportunity to do this placement at Crockers and for the trust he placed in me and in the work. He gave me the freedom to design and build a system that the sales desk would depend on every day, and the backing to work closely with IT and sales to get it right. His support made it possible to bring a real business problem into a capstone, and to see the result go into production during the placement.

I owe a great deal to **Hamida Khan** in IT. She configured Azure and the platform from the ground up, set up the deployment workflows, and was always ready to help when something needed rights I did not hold. She also gave her time as the second coder for the research and as the first person to sign off the application in writing. Much of what made this project possible sat on her side of the work, and I am sincerely thankful for her patience and generosity.

I am equally grateful to **Laura Staallekker** and **James Cowan** in sales. They were willing to change how they worked every day and to let go of a spreadsheet they had relied on, which is never a small thing to ask of a busy desk. Their honest feedback on every screen, and their trust in using the application with real bookings, are the reason it is in daily use today.

Table of Contents

Declaration of Originality 2
Acknowledgement 3
List of Figures 5
List of Tables 6
1. Introduction 7
1.1. Background and context 7
1.2. Problem statement 8
1.3. Objectives 9
1.4. Scope and limitations 10
1.5. Structure of the report 11
2. Literature review 11
2.1. Search, selection, and what the record contains 11
2.2. What the retained studies can support 15
2.3. The contradiction the second search made sharper 16
2.4. What the gap asked of the build 17
3. Methodology, design, and architecture 18
3.1. Research design 18
3.2. How the development record is coded 19
3.3. Validation 20
3.4. Architecture and system boundaries 21
3.4.1. User interface and main screens 23
4. Implementation and development process 29
4.1. System design 30
4.1.1. Server-side booking rules 30
4.1.2. Data ownership 30
4.1.3. Main runtime flows 31
4.2. Software engineering principles in the build 32
4.3. Delivery against the plan 34
4.4. How Cursor was used 35
4.5. Two corrections against a sales rule 36
4.6. Bookings without an assigned unit 37
4.7. Status of the delivered build 38
4.8. Hypercare: removing an app-sent booking 38

5. Ethical and cultural considerations 39
5.1. Personal information and the Privacy Act 39
5.2. Consent, the dual role, and the transcripts 40
5.3. Bias in the coding and in the model 41
5.4. Cultural context and inclusion 42
6. Testing and evaluation 43
6.1. What was planned 43
6.2. What was run 44
6.3. What the script cannot show 44
7. Results and discussion 45
7.1. RQ1 — task types and Cursor outcomes 45
7.2. RQ2 — human control points 49
7.3. Product validation 51
8. Conclusion and future work 53
8.1. Approximate development effort (LOC-based comparison) 53
8.2. Limitations 55
8.3. Future work 56
References 57

List of Figures

Figure 1 - Booking 23
Figure 2 - The Pipeline 25
Figure 3 - The Check-out Calendar 25
Figure 4 - Conflicts 26
Figure 5 - Administration 27
Figure 6 - Tenant Booking, opened from the Booking page. 28
Figure 7 - Admin → Delete booking. 28

List of Tables

Table 1 - Sources used across the two literature reviews. 13
Table 2 - Technology stack of Park Tower Booking. 22
Table 3 - Software engineering principles applied (Benedetti, 2026b). 32
Table 4 - The six phases as in the Research Development and Plan (Benedetti, 2026b). 34
Table 5 - RQ1 coding of all sixty non-merge commits in the development window. 46
Table 6 - Second-coder agreement on the 13 October RQ1 sample 48
Table 7 - Human control points retained during LLM-assisted development. 49
Table 8 - Scripted product scenarios on Dev against the Palace sandbox. 52
Table 9 - LOC-based development/integration effort ranges from five AI sources 54

1. Introduction

This report covers my fourteen-week industry placement at Crockers Property Group, which ends with Week 14 on 3 November 2026. Park Tower Booking, the system I built during the placement, is already in production for the sales desk, and Crockers IT run the Azure environment it sits on. The Research Development and Plan set out the study, and the Work in Progress report, written in Week 13, described the system in daily use and narrowed the research questions. This final report brings the literature, design, build, ethics, testing and results together and closes the study (Benedetti, 2026b).

1.1. Background and context

Crockers Property Group is a privately owned New Zealand property business based in Auckland. It works across residential and commercial property management, body corporate management and build-to-rent, and traces its practice back to 1971. This capstone sat in a small corner of that operation. Park Tower is a single building whose rooms are booked by the sales staff. IT look after the Microsoft estate, including Azure and sign-in, and the group already used Palace as its customer relationship management system (CRM) for tenancies. I worked as the developer on placement for fourteen weeks, with Nick Hofmann as host supervisor (Benedetti, 2026b).

Before this project, every room booking at Park Tower was entered twice: once in Palace, and again in an Excel spreadsheet that fed a Power BI report. Sales kept both copies up to date by hand, and whenever one changed without the other the two records drifted apart. The operational goal was a single place to enter a booking, with Palace still the system of record for tenancies and the spreadsheet retired.

I built Park Tower Booking to do that. It is a Flask application hosted on Azure that holds the booking pipeline, the check-out calendar and the cleaning jobs, and reads and writes tenancies through the Palace REST API. Cursor was my coding assistant for the whole build, from routes and services to front-end modules and the availability and conflict logic. The product rules came from sales, and the decision to merge any change stayed with me. The application itself never calls a language model (Benedetti, 2026a; Benedetti, 2026b) .

The research is built on top of that work. Most published evaluations of coding assistants ask whether generated code passes a test written in advance (Chen et al., 2021; Hou et al., 2024). Many of the rules here had no such test. The clearance rule, for example, which frees a unit only from the second day after check-out, was something sales knew long before anyone wrote it down. The study therefore looks at what Git and the Cursor session files show about the work, and at which human checks kept the rules with the people who own them.

1.2. Problem statement

Two problems run through the project, and the report keeps them apart.

The business problem was double entry. Each booking lived in two places, Palace and the spreadsheet, and the report built from them had to be rebuilt by hand. Park Tower Booking replaces this with one local pipeline: a stay is written in the application first and only reaches Palace when sales send it, while check-outs and cleaning jobs stay in the application. When the two systems disagree, Palace wins on the identity of the tenancy, and Park Tower wins on the pipeline calendar and the cleaning state (Benedetti, 2026b).

The research problem is what that development record can tell us about Cursor. Becker et al. (2025) found that experienced developers working on their own repositories were 19% slower with an assistant, even though they believed they had been about 20% faster. A

smooth release could easily be read as proof that Cursor made the work faster or safer, so the study keeps two strands separate. Validation asks whether the application works for IT and sales; the questions about the assistant are answered from the artefacts (Benedetti, 2026b).

1.3. Objectives

The six objectives set in the Research Development and Plan still stand. Field research with IT and sales produced the rule set and the acceptance criteria, and the literature review was extended in the Work in Progress report. Park Tower Booking is in production with a single booking entry, conflict rules, cleaning jobs, Palace integration and deployment on Azure. IT run the platform, and sales use the screens every day against live Palace data (Benedetti, 2026b).

The development record has been coded against both research questions (sections 7.1 and 7.2), and the scripted validation and UI walk are complete (section 7.3). IT gave written acceptance on 22 October 2026 and Sales on 23 October 2026. The System Usability Scale (SUS) results are also reported in section 7.3 (Brooke, 1996): three responses from Sales, with a mean of 80.0, and one from IT, at 97.5. No further SUS responses were collected. None of the test bookings used for the script were sent to live Palace.

Both questions were narrowed in the Work in Progress report so that each claim could rest on an artefact.

RQ1: Where Cursor involvement is identifiable in Git history and session transcripts, which development task types show acceptance, amendment, or later rework, and in what proportions?

RQ2: Which human control points were retained during LLM-assisted development, and what evidence indicates their role in maintaining ownership of product rules and correctness?

Release readiness sits alongside these questions as validation rather than as a third research question. It covers a complete Palace load, a booking that can be sent, cleaning jobs that follow the rule, and sign-off from both IT and sales (Benedetti, 2026b).

1.4. Scope and limitations

The case is one building, one developer and one assistant. The development window runs from 25 August to 31 October 2026, and in that period the repository holds 74 commits, 14 of which are merges. Of the 60 non-merge commits, 55 are mine and five belong to an IT engineer and deal with the Azure workflow or gunicorn; Table 5 lists all of them (Benedetti, 2026b). The Cursor session files go back to 4 August 2026, before the code moved into the Crockers GitHub organisation. When it moved, on 25 August, the earlier build arrived as a single commit, 2cc393d (Table 5, item 2), with 62 files and roughly 10,600 lines, so those first weeks can only be read from transcripts, not from separate diffs. The last commits in the coded corpus are from 13 October 2026 (Table 5, items 59 and 60). One later hypercare commit, b424044 on 22 October 2026 (section 4.8), falls inside the same window. I describe it in the report but did not add it to Table 5, because there was no time left to recode the sixty items, redraw the second-coder sample or repeat the scripted checks with IT.

The study does not measure whether I worked faster with Cursor. It does not analyse resident records, and it does not treat my own impression of the tool as a finding. Suggestions that left no transcript, and edits I typed straight into a file, are outside what RQ1 can see. A randomised comparison was not possible either: the team is too small and the system was already being built (Becker et al., 2025; Runeson & Höst, 2009).

The ethical and cultural questions raised by the tenancy data, including what the Palace send does with names, school, country and gender, are discussed in Section 5. They are part of the capstone, but they sit outside the research questions (Benedetti, 2026b).

1.5. Structure of the report

Section 2 reviews the literature and the gap that remains. Section 3 sets out the research design, the coding rules and the architecture, including the screens sales use. Section 4 gives the engineering account: the system design, the principles behind it, what each of the six phases delivered, how Cursor was used, and two occasions when a draft had to be corrected against a sales rule. Section 5 deals with privacy, my dual role and the cultural limits of the build. Section 6 covers the test plan and what the script can and cannot show, and Section 7 reports the results for RQ1, RQ2 and the product. Section 8 concludes with the effort comparison, the limitations of the study and suggestions for further work.

2. Literature review

The review was designed to support this case rather than to map the whole field of coding assistants (Benedetti, 2026a). The Research Development and Plan ran the database search and kept 22 sources. The Work in Progress report then added seven empirical studies through citation chasing, along with a separate set of normative sources on privacy, delivery practice and accessibility. This section brings both reviews together and shows where the evidence still runs out.

2.1. Search, selection, and what the record contains

The search followed the guidelines for systematic reviews in software engineering (Kitchenham & Charters, 2007), using the stages of PRISMA 2020 (Page et al., 2021). Scopus, IEEE Xplore, the ACM Digital Library and Google Scholar were searched for work published between January 2020 and August 2026. Three blocks of terms were combined. The first described the tool, from large language models and Copilot through to Cursor and

agentic coding. The second described the activity, from code generation and repair to review, maintenance and the wider development lifecycle. The third described the setting this study is interested in: case and industrial studies, field studies, developer productivity and brownfield systems.

The inclusion criteria were fixed before screening. A source had to be peer-reviewed, a systematic review, or a field experiment with a published protocol; it had to study an assistant on a software engineering activity and report a measurement and its result; and it had to be available in English or Portuguese. Vendor material and papers without an evaluation were excluded.

Twenty-two sources were retained, thirteen of them peer-reviewed. Grey literature, such as a practitioner note on Cursor (Trofimov, 2025), was kept on record but not used as evidence of effect. Becker et al. (2025) did not come up in the database results and was added by hand. Because the first search leaned towards code-generation terms, a citation chase from Becker et al. (2025), Hou et al. (2024) and Moradi Dakhel et al. (2023) into CHI, OOPSLA, CCS and ICSE added seven more empirical studies under the same criteria. None of the seven was set in a brownfield system with rules owned by stakeholders. Statutes, standards and textbooks were added for the ethics and engineering discussion, but they are not part of the empirical set (Benedetti, 2026b).

Table 1 combines the two reviews. The Research Development and Plan kept twenty-two sources and tabulated ten primary studies, and the Work in Progress report added seven empirical studies and the normative sources used for delivery, privacy and inclusion. Five sources from the first review were cited in the text without having a row of their own, so they appear as rows here: Hou et al. (2024), Gomes et al. (2026), Rao et al. (2025), Trofimov (2025) and Jiang et al. (2021). Method guides are not findings, so Kitchenham and Charters

(2007), Page et al. (2021), Braun and Clarke (2006), Runeson and Höst (2009), Hevner et al. (2004), Brooke (1996) and Forsgren et al. (2018) are cited in the method rather than listed in the table.

Table 1 - Sources used across the two literature reviews.

Work	Origin	What it contributes	Limit for this project
Chen et al. (2021)	Plan	Benchmark. 28.8% pass@1 and 70.2% pass@100 on HumanEval.	Isolated functions, with no surrounding system.
Qian et al. (2024)	Plan	Benchmark. 86.66% executability. Removing role prompts dropped a quality score from 0.3953 to 0.2212.	Greenfield tasks, built from scratch.
Hong et al. (2024)	Plan	Benchmark. 85.9% pass@1 on HumanEval.	Greenfield. No maintenance of existing code.
Moradi Dakhel et al. (2023)	Plan	Benchmark. Copilot was comparable to humans on fundamental problems: an asset for an expert and a liability for a novice.	Algorithmic problems, not business rules.
Pearce et al. (2022)	Plan	Security scenarios. About 40% of 1,689 generated programs were vulnerable.	Synthetic scenarios, with no production context.
Schäfer et al. (2024)	Plan	Twenty-five npm packages. Median statement coverage from generated tests was about 70%.	Open-source JavaScript, and no business rules.
Konstantinou et al. (2026)	Plan	Generated test suites. About 20% failed to compile and 43% failed at execution.	Measures the tests, not the features that were delivered.
Jiang et al. (2021)	Plan	Cited with Chen et al. (2024). A repair step can improve a draft by adding a check.	The Plan states no separate measured result for this paper. A booking rule has no oracle.
Chen et al. (2024)	Plan	Benchmark. Self-explanation improved accuracy by 2 to 12 percentage points.	The gain needs an executable oracle.
Becker et al. (2025)	Plan	Real repositories. Developers were 19% slower with an assistant and believed they had been about 20% faster.	Experienced open-source maintainers, not an in-house team beside sales.
de Brito and Farias (2025)	Plan	Practitioner survey. Broad adoption, and a positive view across the lifecycle.	Self-report, with no outcome measure.
Hou et al. (2024)	Plan	Systematic review. Industrial evidence is thin beside the benchmarks. Used to name the gap.	A map of published work, not a result from a system with stakeholder-owned rules.
Gomes et al. (2026)	Plan	Systematic review of LLM-assisted empirical software engineering. The same imbalance between benchmarks and industry.	Locates the gap. It does not measure this case.
Rao et al. (2025)	Plan	Engineering practice for LLM-assisted systems is still immature.	No measurement of a booking rule or a live CRM.

Work	Origin	What it contributes	Limit for this project
Trofimov (2025)	Plan	A practitioner account of Cursor on commercial work. Kept in the record of what was found.	Grey literature. Not used as evidence of effect.
Peng et al. (2023)	Work in Progress	Controlled experiment, 95 developers. Copilot users finished a small, fully specified task 55.8% faster.	The task carried little unwritten context.
Ziegler et al. (2024)	Work in Progress	Survey linked to usage telemetry, over 2,000 users. Acceptance rate best predicted perceived productivity.	In RQ1, acceptance cannot be read as correctness.
Barke et al. (2023)	Work in Progress	Grounded theory from observed sessions. Developers switch between acceleration and exploration.	Used to shape the task-type codes. It does not label a commit in this project.
Mozannar et al. (2024)	Work in Progress	Lab study with labelled session time. A large share of the session goes to reading and checking suggestions.	That cost leaves no trace in a commit.
Liang et al. (2024)	Work in Progress	Survey, 410 developers. Output is often dropped because it misses a functional or non-functional requirement.	The nearest published account of a tacit rule, and it is self-reported.
Perry et al. (2023)	Work in Progress	User study, 47 participants. Assistant users wrote less secure code and were more confident that it was secure.	Supports treating generated configuration as untrusted.
Jimenez et al. (2024)	Work in Progress	Benchmark built from real GitHub issues. Evaluation moves into real repositories.	Every task still ships with tests that define the right answer. A booking rule has no such oracle.
Sommerville (2016)	Work in Progress	Textbook. The design approach follows the problem, and delivery is iterative.	Practice guidance, not a finding about coding assistants.
Humble and Farley (2010)	Work in Progress	Continuous delivery and configuration management.	Practice guidance, not a finding about coding assistants.
Privacy Act 2020 (New Zealand Parliament, 2020)	Work in Progress	Information privacy principles for the tenancy data the import and the send touch.	A legal duty. Not evidence about Cursor.
Human Rights Act 1993 (New Zealand Parliament, 1993)	Work in Progress	Prohibited grounds of discrimination, used to read the fields the app holds.	The Act does not decide whether those fields should be collected.
Office of the Privacy Commissioner (2022)	Work in Progress	Rental-sector privacy obligations, and what the Commissioner observed in practice.	Sector guidance, not a result from this building.
Association for Computing Machinery (2018)	Work in Progress	Duties on the people in the study and on the transcripts.	A professional code, not a measure.
Te Mana Raraunga (2018)	Work in Progress	Principles of Māori data sovereignty.	This app collects no ethnicity data. That is not the same as consultation.

Work	Origin	What it contributes	Limit for this project
Kukutai and Taylor (2016)	Work in Progress	The scholarship those Māori data sovereignty principles sit in.	No ethnicity is collected, and none was asked for.
New Zealand Government (2025)	Work in Progress	The Web Accessibility Standard, read as an inclusion question.	The audit is a recommendation at handover. It has not been done.
World Wide Web Consortium (2023)	Work in Progress	WCAG 2.2, the guideline that standard points at.	Not an audit result.

Note. *Plan marks a source retained in the Research Development and Plan* (Benedetti, 2026a). *Work in Progress* (Benedetti, 2026b) *marks a source added in that report. Becker et al.* (2025) *is listed once, with the Plan. The seven later studies are the ones the Work in Progress report added by citation chasing.*

2.2. What the retained studies can support

The 22 sources were coded inductively (Braun & Clarke, 2006). Two early codes, prompt engineering and model choice, were dropped because nothing in the retained set measured them in an industrial setting. Four themes remained, and they all point in the same direction: an assistant can draft code for a bounded task, but it is much less clear what happens when someone has to check that draft against a real codebase and a rule the model was never told about (Benedetti, 2026a).

On isolated tasks the results are strong, but the setting is narrow. Codex reached 28.8% pass@1 on HumanEval (Chen et al., 2021), and multi-agent systems such as ChatDev and MetaGPT achieve higher rates on software built from scratch (Qian et al., 2024; Hong et al., 2024). Moradi Dakhel et al. (2023) found Copilot comparable to humans on fundamental programming problems, an asset for an experienced developer but a liability for a novice. None of this tells us much about a clearance rule that has to work inside an existing schema and a CRM API (Benedetti, 2026a).

Checking the draft is a separate cost. In Konstantinou et al. (2026), about one in five generated tests failed to compile and 43% failed when executed. Schäfer et al. (2024) reported high statement coverage from generated tests, but coverage only shows that the tests ran, not that they checked the behaviour a user needed. Adding a repair step can improve

accuracy, yet someone still has to confirm the result against the original goal (Chen et al., 2024). In this project, that someone was me.

Passing a check is also a weak guarantee of correctness. Pearce et al. (2022) found that roughly 40% of 1,689 programs generated by Copilot in security-relevant scenarios were vulnerable, and Perry et al. (2023) found that people using an assistant wrote less secure code while feeling more confident about it. Jimenez et al. (2024) moved benchmarking onto real GitHub issues, but every task still comes with tests that define the right answer. A booking rule has no such oracle, and Rao et al. (2025) describe the engineering practice around these tools as still immature.

What developers say about the tools is a fourth, weaker line of evidence. De Brito and Farias (2025) report wide adoption and positive views in a survey, while Hou et al. (2024) and Gomes et al. (2026) point out how thin the industrial evidence still is compared with the benchmarks. Becker et al. (2025) measured that gap directly: sixteen experienced developers working on 246 issues in repositories they maintain were 19% slower with an assistant, mostly Cursor, yet afterwards believed they had been 20% faster. This is why I do not treat my own impression of Cursor as a result.

2.3. The contradiction the second search made sharper

The seven later studies help explain a contradiction that was already visible in the first review. Peng et al. (2023) found a 55.8% speed-up on a small, fully specified task, while Becker et al. (2025) found experienced maintainers slower on their own repositories. Both were controlled studies, so the difference lies in the task: Peng's task was new and fully specified, whereas Becker's work sat inside conventions nobody had written down. Park Tower is much closer to Becker's situation. The clearance rule, the vacant-unit display and the cleaning window all lived with sales before they were documented (Benedetti, 2026b).

The same studies show why acceptance rate is a poor measure of success. Ziegler et al. (2024) found that it tracks how productive developers feel, and Mozannar et al. (2024) showed that much of a session is spent reading suggestions, a cost that never appears in a commit. Liang et al. (2024) report that developers discard output that misses a requirement, which is the closest published account of a tacit rule, although it is based on self-report. Barke et al. (2023) distinguish between acceleration and exploration. That distinction influenced the task types used later, but it was not used to label any Crockers commit on its own.

The field can now describe how developers use an assistant on a fully specified task, and how they describe that use afterwards. What I could not find was evidence, drawn from artefacts, of an assistant working where correctness depends on a tacit rule inside a system the organisation already runs, and of where a person had to step in (Benedetti, 2026b). That is the gap this case addresses, within the limits of what one project can support (Hou et al., 2024; Gomes et al., 2026).

2.4. What the gap asked of the build

The review placed three conditions on the build. First, availability and conflict logic should run on the server, where a draft can be checked against a sales rule. Second, RQ1 should report amendment and rework alongside acceptance, because acceptance reflects confidence more than correctness (Ziegler et al., 2024). Third, secrets should stay out of Git and platform settings should go through IT, since generated configuration is a known source of insecure code (Pearce et al., 2022; Perry et al., 2023). These conditions are also the control points that RQ2 records.

3. Methodology, design, and architecture

The project is both a system built for Crockers and a study of how that system was written. This section therefore covers two designs: the research design, which decides what can be claimed, and the software design, which decides where a booking rule lives.

3.1. Research design

The study is an embedded case study (Runeson & Höst, 2009) built around a design-science artefact (Hevner et al., 2004), with field research involving IT and sales. For RQ1, the unit of analysis is the work item, meaning one coherent change, matched to a transcript where one exists. For RQ2, the unit is the control point: a human decision that kept a rule or a correctness check with the team. Validation forms a third strand. It is scored against the criteria agreed in the first field meetings and does not use the RQ1 labels (Benedetti, 2026b). RQ1 is measured as coded outcomes on those work items (task type, assistant move, accept, amend, reject, and attributable rework), with proportions taken only where Cursor is identifiable. RQ2 is measured as whether each of six control types left an artefact, and whether that type recurred.

A randomised trial was ruled out because it would need many developers, a control group and instrumentation this project did not have (Becker et al., 2025). A survey was also rejected as the main source, for a similar reason: people's estimates drift away from what can be measured. The study is a case built from artefacts, plus a check on the running system. Since I was both the developer and the researcher, the codebook cannot remove that bias, but it does put my judgement in a form someone else can challenge (Benedetti, 2026b).

At the start of the placement I met IT to discuss Azure and the integrations, and sales to walk through a typical booking day. Between them, those meetings covered the booking and

check-out lifecycle, the cleaning rules, reporting and the Azure setup. I coded the notes thematically into a single rule set (Braun & Clarke, 2006), which became the baseline for acceptance, and later review rounds checked the screens against it. Without criteria agreed in writing, validation would have depended on my judgement alone (Benedetti, 2026b).

3.2. How the development record is coded

The coding framework was fixed before the analysis began. Task type is read from the diff and is one of rule logic, integration, user interface, configuration and deployment, documentation, or defect fix. The label comes from the files in the diff, not from the commit message (Benedetti, 2026a). Rule logic is availability, occupancy, clearance, Vacant or Blocked, and conflict checks. Integration is the Palace client, mapping, or another external API. User interface is screens, templates, or front-end behaviour. Configuration and deployment is Azure workflows, gunicorn, Easy Auth, or environment settings. Documentation is a README, a manual, or a commit that only records the system. Defect fix is a change whose purpose in the diff is to correct a named failure, not to add a rule or a screen. If a commit mixes those types, it is Unclassified and counted. Defect fix is in the framework; in the sixty-item corpus it was not used as a sole label.

Whether Cursor drafted or revised the code can only be coded when there is a transcript; without one, the assistant move is recorded as none or unknown, meaning the record is silent.

Accept means the change was merged without any material change to logic, mapping or a rule, and Amend means it was merged after such a change. Accept and amend are mutually exclusive. Reject and attributable rework are flags: they can sit on an accepted or an amended item. Reject requires a transcript showing a proposal that was not used. Attributable rework is a later change to the same logic that can be tied to a defect, a failed scenario or a named

request. Items the artefacts cannot support are marked unclassified and still counted (Benedetti, 2026b).

RQ2 uses six types of control point, and a point only counts if it is backed by a commit, a transcript excerpt or a test incident. The six types are: a rule stated before the assistant wrote code, review of the draft against that rule, placement of the rule on the server, a secret refused, work routed through IT, and stakeholder sign-off. A type is considered recurring only if it appears in three or more work items, or in three clearly separate artefacts when the line has no item in the sixty. Daily use of the system does not count as sign-off, and a statement such as “I always review” is not treated as evidence (Benedetti, 2026b).

The RQ1 corpus consists of the sixty non-merge commits from 25 August to 13 October 2026 (Table 5), ending at commit 3ddff64. The calendar window was kept open until 31 October in case later work on the desk needed to be described. One such commit did appear, b424044 on 22 October 2026 (section 4.8), and it is described in the report rather than added as a sixty-first row. Cursor sessions from before the repository existed are not counted as separate items. On 13 October 2026, every fifth item, starting with item 4, was drawn for a second coder, giving a sample of 12 items (20%). Six of those items came with a matching transcript, translated literally into English, and six did not. The second coder never saw my labels, so the agreement tests the codebook itself. The RQ1 proportions come from my coding of all sixty items (Table 5) and are reported in Section 7.

3.3. Validation

Validation asks whether the application meets the Phase 1 criteria. Twelve scripted scenarios cover the Palace load, the clearance rule, creating a local booking, sending and updating a booking in the Palace sandbox, removing a stay that was never sent, the cleaning flag, an overlap, the cleaning export, a question to sales about the old spreadsheet, and

blocking access to administration. They were run on app-parktower-booking-dev. A pass on a laptop alone does not count, and no test bookings were written to app-parktower-booking-prod. The application has no Cancel booking control: a local row that has not been sent can be deleted from the Edit screen, but deleting a row that has already been sent does not undo anything in Palace. An Admin tool that removes an app-sent row from the Pipeline only was added on 22 October 2026 (section 4.8). Scenario S7 still treats a sent stay as locked for editing and as not voided in Palace.

One sales colleague walks through the screens. Once acceptance has been given, IT and sales staff each complete the System Usability Scale on their own. The target is a mean above 70 in each group, and the mean is reported whether or not it reaches that mark (Brooke, 1996). Sign-off takes the form of an email from each group, and a reply that points out a gap still counts. If a check-out scenario fails, it is recorded as a failure, no matter how clean the Cursor history looks (Forsgren et al., 2018).

3.4. Architecture and system boundaries

Park Tower Booking is a modular monolith, deployed as one App Service per environment (Sommerville, 2016). Flask blueprints handle HTTP requests, services hold the booking, cleaning and conflict rules, and repositories handle SQL and the Palace client. Because the rules kept changing during user review, splitting the system into microservices would have added cost without any real benefit. The browser can grey out a date, but refusing a double booking, or adding a row to Conflicts, is done on the server (Benedetti, 2026b).

Palace remains the authority on properties, units and tenancies. The application imports those records and, when sales send a booking, inserts or updates the tenancy so the CRM stays current. The send writes school, country and gender into the Palace notes, requires both a first and a last name, writes the last name in capitals, and puts the school in

front of the first name. What this does to names that do not follow that pattern is discussed in Section 5. Check-outs and cleaning jobs stay in Park Tower, and the spreadsheet is no longer used as a booking record (Benedetti, 2026b).

Clearance follows the rule sales already used: a stay that ends on day D frees the unit for a new entry on day D+2, so a stay ending on 25 September frees it from 27 September. The check lives in clears_before_entry in app/models/occupancy.py, with CLEARANCE_DAYS set to 2, and conflict detection uses the same server-side dates from Palace, the pipeline and the cleaning jobs.

The code runs on two App Services: app-parktower-booking-dev follows the dev branch and app-parktower-booking-prod follows main. They are separate applications, not deployment slots. GitHub Actions, using OpenID Connect, builds and deploys every push and leaves local environment files out of the package (Humble & Farley, 2010). I can trigger that pipeline myself, but App Settings, Entra ID and any schema change that needs elevated rights go through IT. Sign-in uses Easy Auth with Entra ID, and a row in the AppUsers table decides who can use the application and who can open administration. Secrets are kept in an ignored .env file locally and in App Settings on Azure (Benedetti, 2026b).

Table 2 - Technology stack of Park Tower Booking.

Concern	Choice
Language and pages	Python 3, Flask 3, Jinja2, plain ECMAScript modules
Process	gunicorn on Azure App Service
Data	Azure SQL, reached through pyodbc
CRM	Palace REST API, reached through requests
Exports	openpyxl, for the cleaning workbook and other Excel output
Identity	Easy Auth, Microsoft Entra ID, and the AppUsers table
Delivery	GitHub Actions with OpenID Connect; dev branch to the development App Service; main to production
Secrets	Ignored .env locally; App Settings on Azure

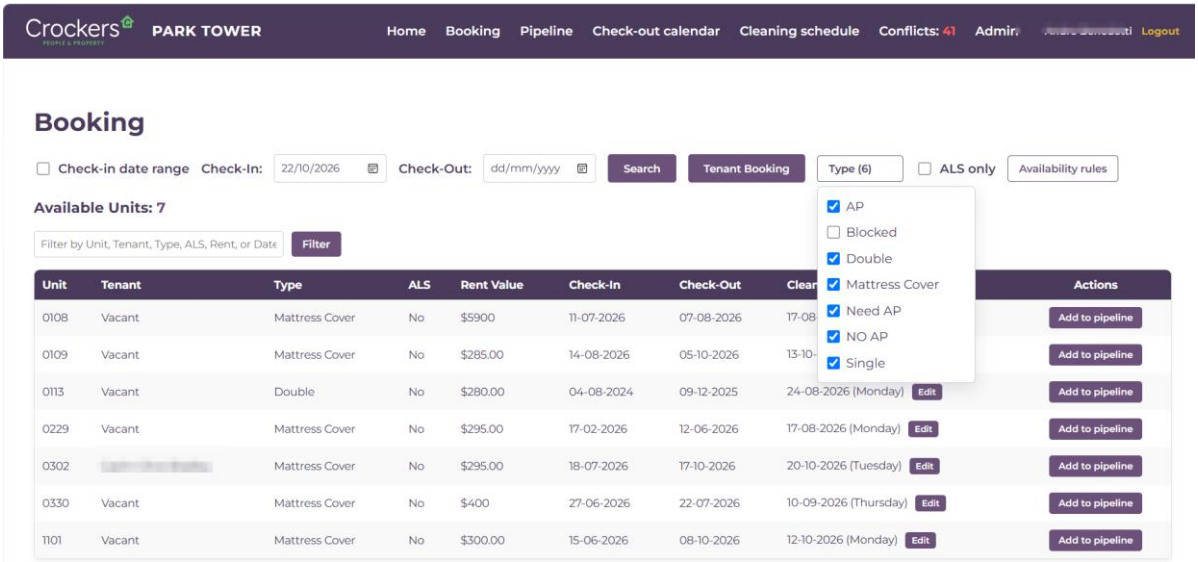
3.4.1. User interface and main screens

The same design can be seen in the screens. A request from the desk crosses four boundaries. The browser collects the dates and displays whatever the server returns; it never decides on its own that a room is free. Booking (called Find available units until Week 12) asks the availability service, which reads Palace tenancies and local pipeline rows, applies the clearance check and returns the list. Conflicts work the same way: the screen lets staff filter by type, but detection happens on the server.

The Palace boundary is drawn differently. The import reads tenancies through a hybrid GET that combines v2ViewAllDetailedTenancy with v2ViewAllDetailedTenancyByProperty, and a send writes a single tenancy back, only when someone presses Send to Palace. Until then, a local booking is invisible to the CRM. Check-out dates and cleaning jobs never cross this boundary; they stay in Park Tower, and the cleaning providers receive them as an Excel file rather than as Palace tenancies. Identity is handled at the edge of the App Service, where Easy Auth accepts an Entra ID sign-in and AppUsers decides whether that person may open administration. Deployment has its own boundary next to it. GitHub Actions can push a new build to either App Service, but it cannot create an App Setting, register an application in Entra or change a schema that needs elevated rights. Those changes wait for IT.

The screenshots below were taken on app-parktower-booking-dev on 22 October 2026, after the Week 12 rename to Booking and the hypercare Admin delete (sections 4.6 and 4.8). Names, the signed-in operator and email addresses are blurred, while unit numbers, dates and statuses are left readable. Occupancy and the Check-in report are buttons on the Pipeline page rather than navbar items. The navbar itself contains Home, Booking, Pipeline, Check-out calendar, Cleaning schedule, Conflicts and Admin.

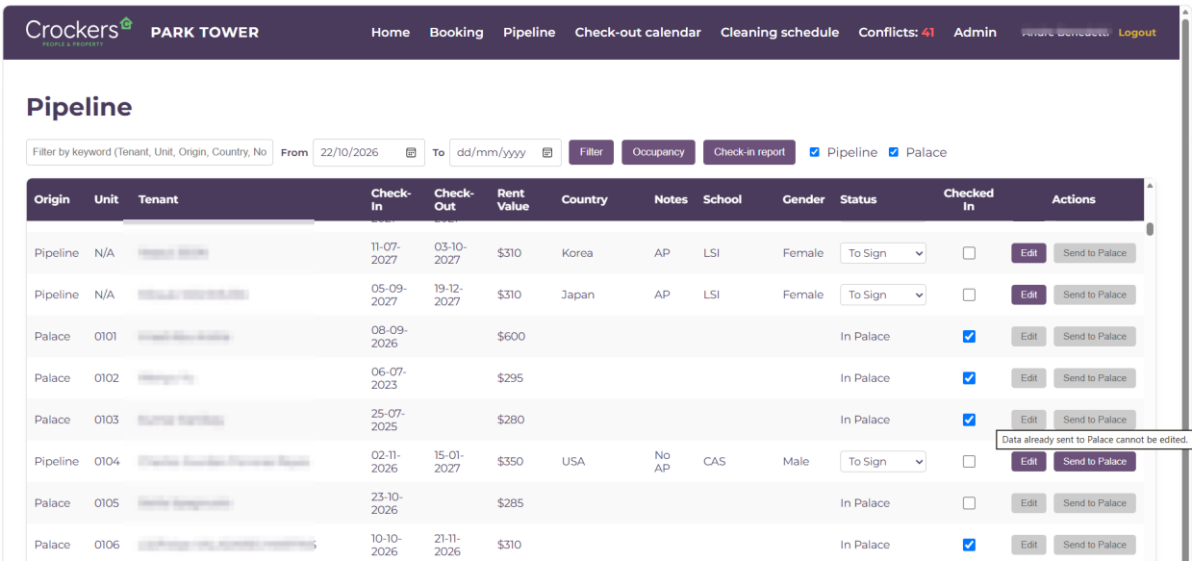
Figure 1 - Booking



Note: Tenant Booking sits beside Search and is not a navbar item. The type filter leaves Blocked out unless the search asks for it. Vacant is the display rule for a tenancy that ended more than a week before the requested entry.

A check-in date is enough to ask which rooms are free, and the list is worked out on the server, not in the browser. The availability service applies the clearance rule, so the previous stay must have ended at least two days before the new entry (Benedetti, 2026b). Choosing Add to pipeline on a unit in the grid writes a local row without calling Palace. Tenant Booking opens the same form without a unit (PropertyCode is stored empty; Figure 6) and writes a row that only appears on Pipeline, as Unit N/A. Send to Palace stays disabled for that row until Edit → Change unit assigns a room. The Vacant label applies the display rule stated in August, which a draft dropped and which was restored on 10 September: a unit is shown as vacant when the previous check-out is more than seven days before the suggested entry. Blocked rooms are left out of the default type filter. Blocked is master data set in administration, and the search only reads it.

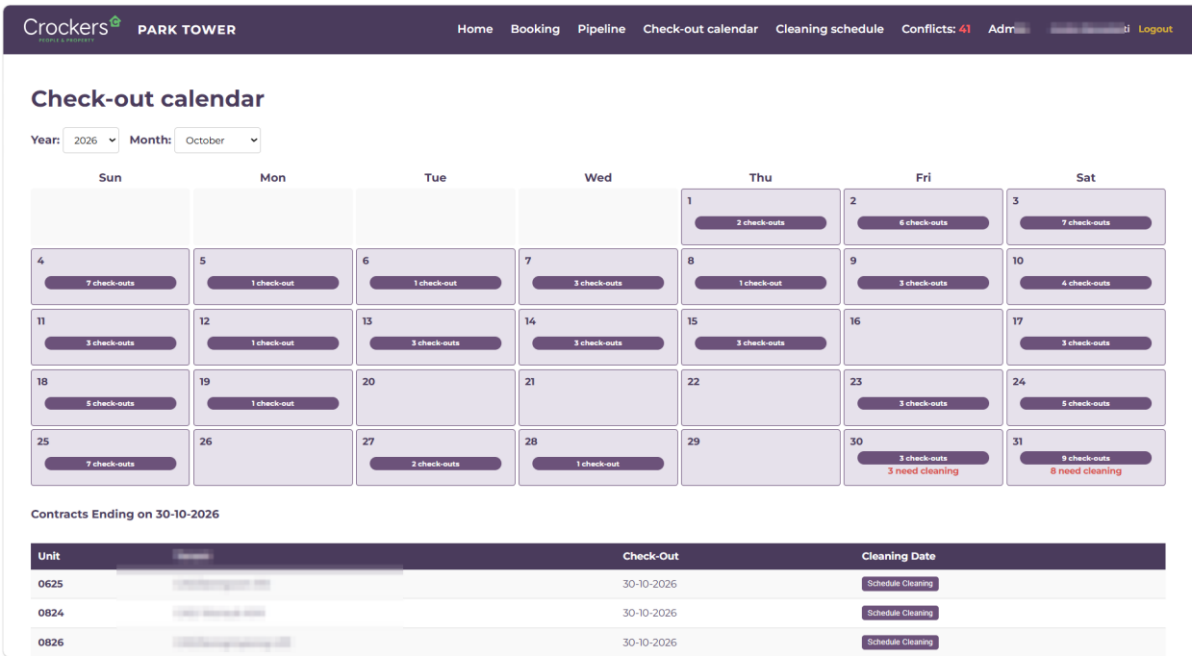
Figure 2 - The Pipeline



Note: It merges Palace tenancies with local bookings. Origin is Palace or Pipeline. Two roomless Tenant Booking rows show Unit N/A. A sent row keeps Edit disabled (Can only be edited in Palace). School, country and gender are on the row, and the send copies them into the Palace notes.

The Origin column shows where each row came from, and the filters offer only Palace and Pipeline. A Palace row is a tenancy that already exists in the CRM. A Pipeline row was created in the application; this includes roomless Tenant Bookings (Unit N/A) and rows already marked SENT_TO_PALACE. Until Send to Palace is used, a stay is purely local, so Delete in the Edit modal removes only that unsent row and never calls Palace. Once a row is SENT_TO_PALACE, Edit and Send are disabled and the Edit modal, with its Delete button, no longer opens. Removing such a row from the application is an administrator action (section 4.8), and even then Palace is not voided: if the hybrid GET still finds the tenancy, Pipeline shows it again as a Palace row. The school, country and gender columns are the fields discussed in Section 5. On this screen they are operational data, but they are also personal information with a narrower purpose than a name and two dates. Check-in state and status are shown on the same row, so the desk sees a single stay instead of the old split between the CRM and the spreadsheet.

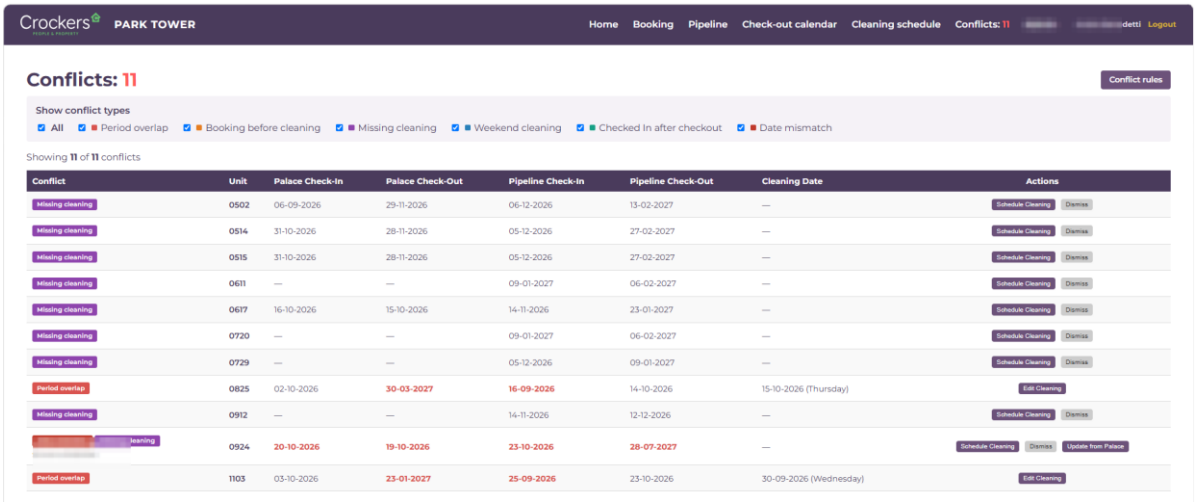
Figure 3 - The Check-out Calendar.



Note: A day is marked when a check-out still has no cleaning job. Cleaning jobs stay in Park Tower. They are not Palace tenancies.

The calendar turns a rule that sales applied long before it was coded into something visible: every check-out should trigger a clean. The red count shows missing cleaning jobs, not a Palace status. Opening a day lists the units concerned, and a clean can be scheduled from there. Each job then moves from New to Sent to Cleaner to Completed, and Export New produces the file the cleaning providers receive. That file replaced a second round of typing without creating a second booking record.

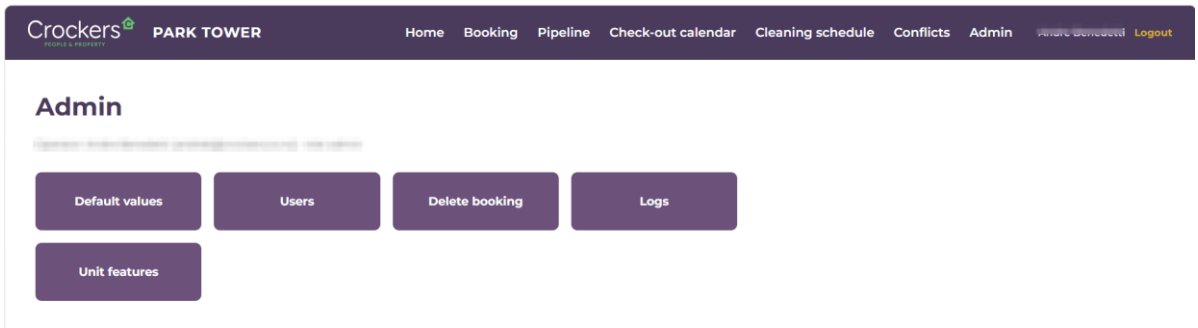
Figure 4 - Conflicts



Note: Conflicts detected on the server from Palace dates, pipeline dates and cleaning jobs. The navbar count is this list. A grey cell on Booking is not the control that refuses a clash.

The conflict types on this screen come straight from the rule set agreed in the field meetings: overlapping periods, a booking before cleaning has finished, a missing clean, a weekend clean, a check-in after check-out, and dates that do not match Palace. Staff can schedule the clean, dismiss a row where the rule allows it, or copy the Palace dates onto the local row when the local copy is the one that is wrong (Benedetti, 2026b). Dismissing a booking-before-cleaning row, or rejecting a missing-clean flag because a clean was already scheduled within the turnover window, is exactly the kind of review RQ2 sets out to record. The screen is where the rule becomes visible, but the decision is still made on the server.

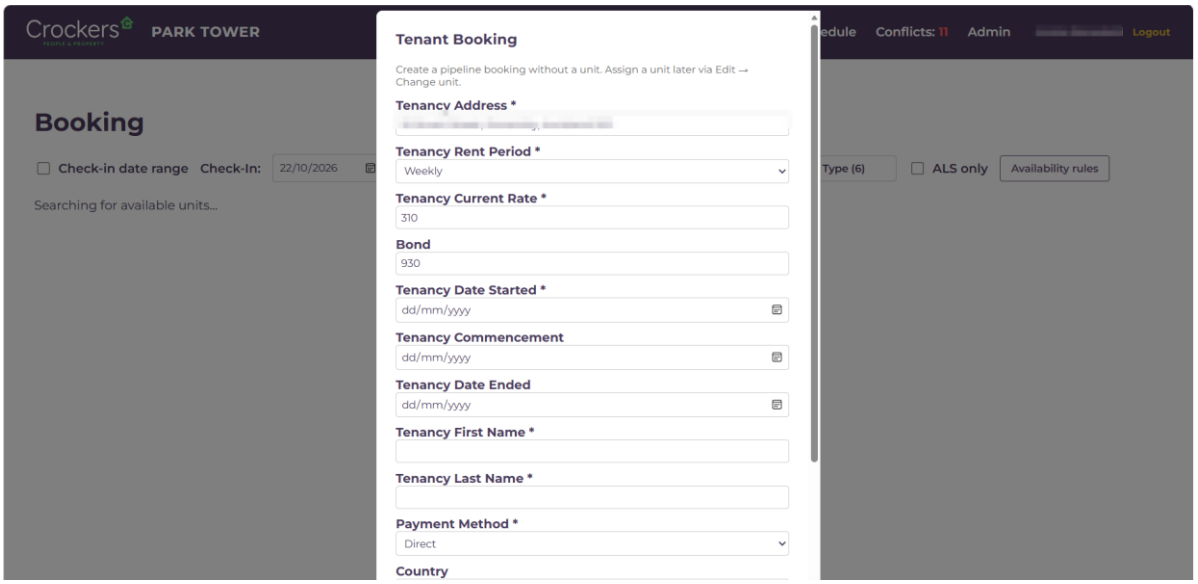
Figure 5 - Administration



Note: Admin is limited to an AppUsers administrator. The landing order is Default values, Users, Delete booking, Logs, and Unit features.

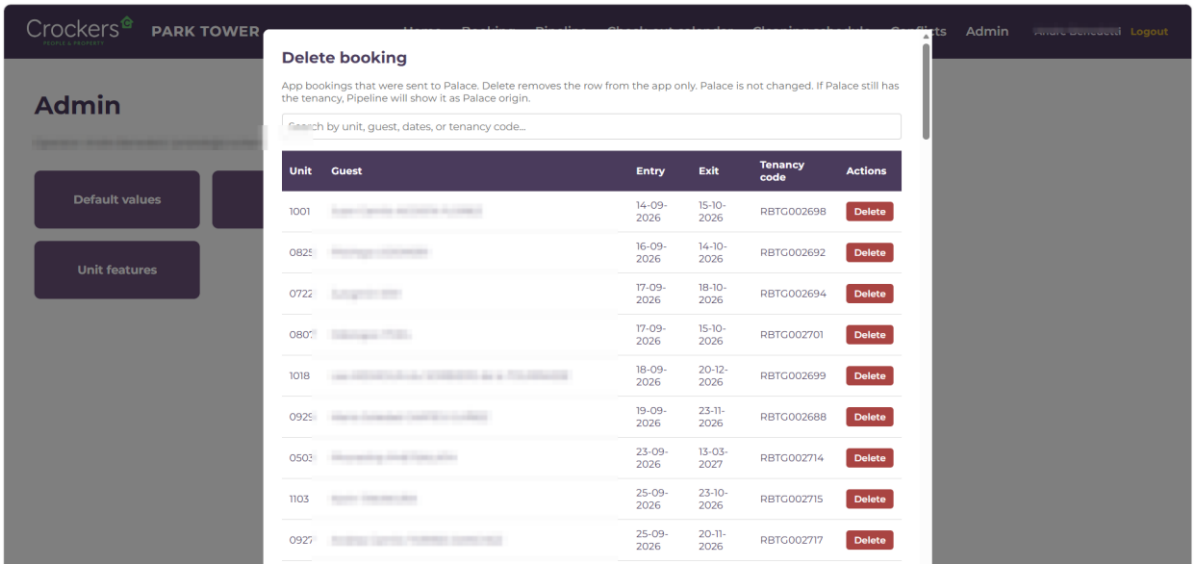
Signing in with Entra ID does not make someone an administrator; that role is held in AppUsers. Default values control the next booking form, including the type filter that leaves Blocked rooms out (the defaults screen still calls this control Find available units — Type filter default). Unit features is where a room is marked as Blocked or given an ALS (available for language school) flag, which Booking then reads. Delete booking lists SENT_TO_PALACE rows and removes only the local copy (Figure 7; section 4.8). The activity log records material changes, such as a Palace send, a change in cleaning status, a dismissal or an administration edit, and it can be exported. That log is also a trail RQ2 can rely on when a commit message is only a few words long. A signed-in user who is not an administrator is turned away from the administration routes, which scenario S12 checks.

Figure 6 - Tenant Booking, opened from the Booking page.



Note: The form is Add to pipeline without a unit. The row is written locally; Send to Palace stays disabled until Edit → Change unit assigns a room.

Figure 7 - Admin → Delete booking.



Note: The list is app rows already sent to Palace. Delete removes the local row only; Palace is not voided. If the hybrid GET still finds the tenancy, Pipeline shows it again as Palace origin (section 4.8).

Delivery happened in short increments shown to sales on the development App Service, and their feedback decided the next change. There is no automated test suite. Scripted scenarios and acceptance were enough to put the application into daily use, but on their own they would not have caught a clearance rule that had to be corrected twice in September. A small set of tests around that rule is one of the handover recommendations (Benedetti, 2026b).

4. Implementation and development process

This section gives the engineering account of Park Tower Booking: the system design, the principles behind it, what the six phases delivered, how Cursor was used during the build, and two corrections that show the limits of a plausible draft. The screens in Section 3 showed the system from the desk; here the question is why each piece sits where it does.

4.1. System design

The design had to suit a small team, a set of rules that kept growing after the first booking, and a CRM that needed to stay current. Park Tower Booking is therefore a modular monolith: one codebase and one App Service per environment (Sommerville, 2016). Flask blueprints handle HTTP, a thin service layer holds the booking, cleaning and conflict rules, and repositories handle SQL and the Palace client. With a small set of entities and rules that kept changing until Week 9, microservices or a heavy domain model would have added cost for no real gain. The object model is deliberately light. Dataclasses carry the records, and repository classes keep SQL and HTTP away from the services, so a change to a Palace field does not have to be repeated on every screen (Benedetti, 2026a).

4.1.1. Server-side booking rules

Availability and conflict checks run on the server. They combine Palace tenancies with local pipeline rows and apply the clearance rule (D+2; section 3.4) through clears_before_entry in app/models/occupancy.py, which the availability service calls. The browser may grey out a date, but it is not trusted to refuse a double booking; that refusal, or a new row on the conflicts screen (Figure 4), comes from the server. Keeping the rules in the service layer means there is one place to look when something is wrong. It is also where a plausible but wrong suggestion does the most damage, which is why the correction of 22 September had to go into occupancy.py. Fixing only the page would have left the API free to save the wrong date (Benedetti, 2026b).

4.1.2. Data ownership

Palace stays authoritative for properties, units and tenancies. The application imports those records, dropping the fields it does not need, and when sales send a booking it inserts or

updates the tenancy in Palace, including the school, country and gender in the notes. Check-outs and cleaning jobs belong to Park Tower, and the spreadsheet is no longer a booking record. When the two systems disagree, Palace decides who holds the tenancy and Park Tower decides the pipeline calendar and the cleaning state. This settled a question the Research Development and Plan had left open: which store would become the source of truth once double entry stopped. Figure 2 shows the same split from the desk. A row marked Palace is already a tenancy, while a row marked Pipeline stays local until someone sends it (Benedetti, 2026b).

4.1.3. Main runtime flows

Four flows cover the sales day, and each stops at one of the boundaries described above. Booking asks the availability service which rooms are free on the requested dates (Figure 1). A pipeline booking is written locally, either from a unit in the grid or without a unit through Tenant Booking, and only reaches Palace when the operator sends it, which a roomless row cannot do. Before the call to Palace the payload is validated, and afterwards the local row is linked to the new tenancy. The check-out calendar and the cleaning schedule read from the same store, and any check-out without a cleaning job is flagged there (Figure 3). Conflict detection is a single server-side pass over Palace dates, pipeline dates and cleaning jobs. Export New reads only jobs still marked New and produces the file for the cleaning providers. Occupancy and the check-in report open from Pipeline rather than from the navbar. None of these flows calls a language model: Cursor helped write the code but plays no part in the running system (Benedetti, 2026b).

4.2. Software engineering principles in the build

The principles in Table 3 were chosen for this particular problem rather than taken from a checklist. The team was small, the rules belonged to sales and were often only spelled out once a screen showed something wrong, Palace could not be allowed to drift away from the booking desk, and I did not hold the rights to change Azure, Entra ID or the database schema. A principle that did not help with at least one of these facts was left out.

Table 3 - Software engineering principles applied (Benedetti, 2026b).

Principle	Where it shows up	What it did
Separation of concerns and layering	Routes, services and repositories. The screens in Section 3.5 are the HTTP edge.	A rule change from sales stayed in the services. A change to a Palace field stayed in the Palace client.
Single source of truth	Palace owns tenancies. The application owns the pipeline calendar and the cleaning jobs.	The spreadsheet ceased to be a second booking record, and the application did not become a second CRM.
Defensive design	Server-side availability and conflict checks. The payload is validated before any Palace send.	A double booking is refused whatever the browser shows. An incomplete payload does not reach Palace.
Iterative, incremental delivery	Short increments on the development App Service, with sales review through Week 9.	Late rules, including vacant units, dismiss and unit features, were absorbed without a new project plan.
Continuous delivery and configuration management	GitHub Actions. The dev branch feeds one App Service and main feeds the other. App Settings, and an ignored .env file.	A deployment can be repeated. Secrets stay out of Git. IT can own the pipeline (Humble & Farley, 2010).
Least privilege	Easy Auth plus the AppUsers table. The SQL login cannot change the schema.	An unregistered sign-in cannot enter. A person who is not an administrator cannot open Figure 5. A schema change needs IT.
Traceability	The activity log of material changes, with a CSV export.	A Palace send, a dismiss or an administration edit can be traced afterwards. That trail is also evidence for RQ2 when a commit message is only a few words.

Delivery followed these increments rather than a named method such as Scrum. Sales used the development App Service, and whatever they rejected became the next change. That worked for one booking desk and one developer, but I am not suggesting it would suit a larger programme.

The layering is what made it safe to show sales each increment. A vacant-unit rule changes how the availability service chooses rows for display; it does not require changes to the page, the SQL and the Palace client at the same time. Clearance works the same way: once clears_before_entry is the only check, any later edit shows up in one file. Most of the Cursor work on availability happened in that layer, which is also why an amendment there matters more than an amendment to a label.

Least privilege and continuous delivery turned out to be two sides of the same arrangement. I can start the pipeline that deploys a build, but I cannot create an App Setting, register an application in Entra ID or apply a schema change that needs elevated rights. The five commits in the study window made by an IT engineer (Table 5, items 7, 10, 32, 33 and 34) are the deployment workflow and the gunicorn dependency, so generated configuration was never shipped just on my say-so. This was deliberate. Following Pearce et al. (2022), any draft that touches credentials or the platform was treated as untrusted until the person holding the rights had put it in place.

One principle was under-applied: there is no automated test suite (Benedetti, 2026b). Daily use was supported by the scripted scenarios and the acceptance review with IT and sales, and that was enough to put the application into production before Week 14 ends on 3 November 2026. It would not, by itself, have caught the availability rule being adjusted on 17 September and again on 22 September. The Research Development and Plan advised against expanding the build to close every gap found along the way, so I record the missing tests here as a lesson. The handover recommends a small suite around clearance and cleaning, but that suite is not part of what was handed over.

4.3. Delivery against the plan

The six-phase plan from the Research Development and Plan held up in substance, but not in its timing. Table 4 compares what was proposed with what actually happened. User review ran until Week 9, because sales kept identifying new rules once they could see a screen that was wrong. Cloud set-up started in Week 7, earlier than planned, and was then handed over to IT. The application was in production well before Week 14 ends, while the research coding, the second coder, the sandbox checks and this report continued alongside it (Benedetti, 2026b).

Table 4 - The six phases as in the Research Development and Plan (Benedetti, 2026b).

Phase	Planned weeks	What was planned	How it ran
1	Week 1	Field research with IT and sales, ethics, and an agreed rule set.	Done in Week 1. The rule set is the acceptance baseline.
2	Weeks 2–4	Palace import, data model, and a first end-to-end booking.	Done in Weeks 2–4, on a validated Palace load.
3	Weeks 5–7	User review, and a prototype accepted for deployment.	Review ran to Week 9, on the Azure build. Conflicts, vacant units, cleaning, reporting, unit features and AppUsers were added here.
4	Weeks 8–9	App Service, SQL, pipeline, and production.	Deployment began in Week 7. Both App Services were in place. The platform then sat with IT.
5	Weeks 10–12	Integration, interface checks, acceptance, and the usability scale.	The application was accepted for daily use during the placement. The formal usability summary for this report is still open.
6	Weeks 13–14	Hypercare, coding of the Cursor record, and the final report.	Week 14 ends on 1 November 2026. The application is in production. The research coding and this report continue through Week 14.

Note. Planned weeks are those in the Research Development and Plan. The two moves that matter are Phase 3, which ran long, and Phase 4, which started early. Phase 6 closed the placement. It did not close the study.

Booking returns the rooms that are free under the clearance rule. Tenant Booking writes a local stay without a unit, which appears only on Pipeline as N/A until Change unit assigns a room. The pipeline merges local rows with Palace tenancies and is where stays are sent from. The check-in report replaced the Power BI pack that used to be rebuilt by hand. The check-out calendar flags any day where a check-out has no cleaning job, and Export New produces the list the cleaning providers receive. Conflicts are detected on the server for overlaps, bookings before cleaning, missing cleans, weekend cleans, check-ins after check-out, and dates that do not match Palace. Administration is limited to administrators and, since 22 October 2026, includes a local-only delete for bookings already sent from the app (section 4.8).

Several of these screens were only added after the first end-to-end booking, once sales could see a wrong result: conflicts, the vacant-unit display, the cleaning schedule, dismissals, unit features and missing units. Most of the rules behind them had never been written down. Anything that needed elevated rights waited for IT, because the pipeline can deploy code but cannot grant access to the platform (Benedetti, 2026b).

4.4. How Cursor was used

The working sequence was set in the Research Development and Plan. I described the product rule in my own words, Cursor drafted or revised the code, and I reviewed the diff against that rule and the existing code before accepting, amending or rejecting it (Benedetti, 2026b). The commit, together with the session file when one was kept, forms the RQ1 record, and the human steps form the RQ2 record. I used Cursor most on availability and conflicts, where the rules interact and some of the context was never written down, but also on routes,

front-end modules and code review. The running application does not call any model: a search, a Palace send, and a conflict check are all ordinary server code.

The record turned out thinner than the proposal had hoped. Many commit messages are only a few words long, so the diff has to carry the claim (Benedetti, 2026b). The 25 August bulk commit cannot be broken down into assistant moves, and although session files go back to 4 August 2026, only some of them can be matched cleanly to a later commit. Section 7.1 reports how many items could not be classified as a result.

4.5. Two corrections against a sales rule

Two episodes deserve to be described in full. Each has both a transcript and a commit, and each is the kind of miss the literature predicts when a rule is tacit. They are presented here as development history; the proportions across all task types are reported in Section 7.

On 10 September I noticed that the vacant-unit rule had disappeared from the search. The rule, which I had stated on 13 August before Find available units was rewritten, says that if the previous tenancy ended more than a week before the requested entry, the current tenant is shown as vacant. I told Cursor not to drop any rule unless explicitly asked, and the rule was restored in commit 487de75 (Table 5, item 26). The instruction and the diff are the evidence. This was a review against a rule that already existed, not a new product decision made on the spot.

On 22 September the clearance rule was consolidated in one place. Earlier that afternoon I had asked for a one-day gap, so that behaviour followed my own instruction and was not an assistant error. Once sales and I had settled on two days, I asked Cursor to treat clearance as check-out plus two and to accept any entry date that satisfies the rule. The draft set the first free day one day too late: for a stay ending on 25 September, it showed the unit as free only from 28 September, whereas the sales rule frees it on the 27th. I checked the date and rejected

the off-by-one, and the fix in commit 235e93b (Table 5, item 49) placed the check in clears_before_entry on the server rather than in the browser.

A third pattern shows up in the Git history rather than in a single transcript. Five commits in the window were made by an IT engineer: gunicorn for the App Service, and the Azure deployment workflows as they were added, removed and restored (Table 5, items 7, 10, 32, 33 and 34). As section 4.2 explains, I could push code through those workflows but not change the platform itself, so generated configuration only went live once IT had put it in place (Pearce et al., 2022).

4.6. Bookings without an assigned unit

In Week 12, on 13 October 2026, users asked to be able to schedule a guest before assigning a room. The request had several parts: rename Find available units to Booking; add a Tenant Booking action that opens the same form as Add to pipeline but without a unit; show those rows only on the Pipeline, as N/A, while still allowing Edit and Change unit; and keep them off every other page. Before any code was written, they also asked for Send to Palace to stay disabled until a unit had been assigned. The request is recorded in that morning's session transcript (second-coder pack, item 59).

Commit b41d2fb (Table 5, item 59) implements all of this and was merged the same day through pull request #11; the merge commit a142848 is not a separate RQ1 row. PropertyCode is stored as an empty string because the column is NOT NULL. Pipeline lists roomless rows, while the other views filter them out through include_roomless and related checks, so a roomless booking never occupies a unit in availability, conflicts or the cleaning and check-out calendars. On Pipeline, Send to Palace is disabled until Change unit assigns a property code, after which the row behaves like any other local booking. The change needed no schema migration and did not touch the clearance or Vacant rules. An earlier idea from 6

October, an “add room” button, was not what shipped; the final design is N/A plus Edit → Change unit.

In the RQ1 corpus this change is item 59 (UI; Draft; Accept; clear transcript). In the RQ2 log it counts as Rule stated first, because the product rule was written down before Cursor produced the plan and the code.

4.7. Status of the delivered build

By Week 12 the application was already in daily use, and bookings without an assigned unit (section 4.6) were merged on 13 October 2026. Park Tower Booking remains in production under the care of sales and IT: sales use it against live Palace data, and IT run the platform, Easy Auth and any schema change that needs elevated rights. One further hypercare change, made on 22 October 2026, is described in section 4.8 but not coded into Table 5. The handover recommends tests around the clearance and cleaning rules, a written purpose for sending school, country and gender to the Palace notes, and an accessibility audit. None of these was delivered as part of the placement build (Benedetti, 2026b).

4.8. Hypercare: removing an app-sent booking

During hypercare in Week 14, with the application already in daily use, the desk raised a need the Week 12 build did not cover: a booking already sent to Palace sometimes had to be taken off the Pipeline without being voided in Palace. The agreed rule was local-only removal. The Palace writes the application uses, v2InsertTenancy and v2UpdateTenancy, can insert or update a stay but cannot void or delete one, so this change makes no Palace write at all. Reads still go through the hybrid GET (v2ViewAllDetailedTenancy together with v2ViewAllDetailedTenancyByProperty), which means that if Palace still holds the tenancy, Pipeline shows the stay again as a Palace row.

Commit b424044 (22 October 2026) adds an Admin tool, Delete booking, which lists SENT_TO_PALACE rows, asks for confirmation ("Remove this booking from the app only? It will not be deleted in Palace…") and then deletes the row from the Bookings table. If the delete fails, the row stays. Editing a sent row from Pipeline is still blocked ("Can only be edited in Palace"), and scenario S7 is unchanged: there is still no Cancel booking control and no way to void a stay in Palace. I have described the commit here rather than adding it to Table 5, or recoding Tables 6 to 9 and the second-coder sample, because there was not enough time left in Week 14 to recode those artefacts with IT.

5. Ethical and cultural considerations

The placement put a booking system between sales staff and the personal information of the people living in Park Tower. The research questions do not ask whether that was handled acceptably, so this section does. It covers the Privacy Act duties that the import and the send actually trigger, my dual role, bias in both the coding and the model, and the cultural limits of how names and related fields are written to Palace. These limits are recorded here; the build was not redesigned after the placement to remove them.

5.1. Personal information and the Privacy Act

Because the Palace import and the booking send move personal information, the Privacy Act 2020 applies (New Zealand Parliament, 2020). Its principles limit collection to what the purpose requires, call for reasonable security, limit how long information is kept, and restrict how it is used and disclosed. A property manager carries these duties like any other agency, and over-collection is already a recognised problem in the rental sector (Office of the Privacy Commissioner, 2022). In practice, the application keeps just enough of a tenancy to know

who holds a unit and when a check-out should trigger a clean, and drops the other Palace fields at import (Benedetti, 2026b).

Reviewing the data model during the placement, I realised I had understated what the application holds. As well as names, contact details, rent and bond, each booking stores a school, a country and a gender, and the send writes all three into the Palace notes (the columns are visible on the pipeline in Figure 2). Sex and national origin are prohibited grounds of discrimination under the Human Rights Act 1993 (New Zealand Parliament, 1993). Storing these fields is not unlawful in itself, but under the purpose principle each one needs a written, narrow reason, and none of them may affect the price or who is offered a unit. The Week 1 rule set does not yet record that reason. Agreeing it with sales, making fields optional where a booking does not need them, and setting a retention period for local rows were left as handover actions, and they were still open when the placement ended (Benedetti, 2026b).

Keeping that information secure is a separate duty. Secrets are stored in an ignored local file and in Azure App Settings, never in Git. Any generated configuration that contains a credential is treated as untrusted until IT have set the real value, which is how I applied the security findings of Pearce et al. (2022) and Perry et al. (2023) in practice. The activity log can show that a send happened, but that does not justify reproducing a resident's details in this report.

5.2. Consent, the dual role, and the transcripts

Ethics approval and consent from Crockers were in place before IT and sales staff were treated as participants, and they knew they were both colleagues on the booking desk and participants in a capstone. The harder issue was that I was the developer they worked with every day and also the person asking whether the application was fit for use. People tend to

be generous in that situation. To counter this, the study relied on checks fixed in advance: criteria written down in Week 1, review against those criteria, and a free-text comment alongside the usability score, since a kind number from a colleague says less than the sentence written underneath it (Benedetti, 2026b).

Residents are data subjects, not participants, and none of their records were analysed for RQ1 or RQ2. Session files can still contain a pasted row or a name, because that is often how a defect is shown to an assistant (Benedetti, 2026b). No live booking data is quoted in this report, and any excerpt used is redacted; the screenshots in Section 3 were prepared on the same basis for the Work in Progress report. This is the ordinary obligation that applies when publishing material about people who did not agree to take part in the study (Association for Computing Machinery, 2018).

5.3. Bias in the coding and in the model

Coding the record after the event can favour the story I would like to tell, which is that Cursor was comfortable on ordinary screens and weaker where the rules were tacit. To limit this, the framework was fixed before labelling began, unclassified items were counted instead of being forced into a category, and a second coder applied the same rules to a sample I could not choose after the fact. None of this removes my dual role, but it does make my judgement open to challenge.

The model brings its own bias. A draft of availability logic reflects whatever booking practices dominated its training data, and those are neither Park Tower's nor those of New Zealand student accommodation. Fluent code is not evidence that a cleaning window is right, and Perry et al. (2023) found that people using an assistant tend to trust that fluency. The 22 September episode shows the risk in concrete terms: the draft was coherent, yet it freed a unit on 28 September for a stay ending on the 25th, when the sales rule frees it on the 27th.

Catching that depended on my checking the date, not on how convincing the code looked (Benedetti, 2026b).

5.4. Cultural context and inclusion

The system is operated by a small group of Crockers staff, plus the cleaning providers who receive the Excel export. Many of the people in the data are students placed through their schools, and some come from overseas, which is why the application holds a school and a country. This mix also exposes an assumption built into the Palace send. The code requires both a first and a last name, writes the last name in capitals and puts the school in front of the first name. That works for names written as given name followed by family name, but it rejects people with a single name and can put the family name in the wrong field when it comes first, as is common in Chinese, Korean and Vietnamese names. A compound surname, common in Brazil, only reaches Palace intact if staff type the whole surname into the last-name field. Names with macrons, as in many Māori names, were not tested end to end through Palace and the exports (Benedetti, 2026b).

These are limitations of the build that was left in production. I am not changing the send now that the placement is over, and this report does not propose a redesigned name form. When a name does not fit, sales staff work around it, usually without any record of what they did. The interface has not been audited for accessibility either (Benedetti, 2026b). As part of the handover I will recommend to Crockers an audit against the New Zealand Government Web Accessibility Standard and WCAG 2.2 (New Zealand Government, 2025; World Wide Web Consortium, 2023), but recommending an audit is not the same as having done one.

Some residents may be Māori, and under Te Tiriti o Waitangi information about Māori is more than an operational record (Te Mana Raraunga, 2018; Kukutai & Taylor,

2016). The application collects no ethnicity data and produces no tenant analytics, which keeps the collection small, but that is not the same as having asked Māori about it. Any future report that broke bookings down by country, or by any other demographic field, would need that conversation before it was built. No such report is part of this system (Benedetti, 2026b).

6. Testing and evaluation

6.1. What was planned

The product check is separate from RQ1 and RQ2. It asks whether Park Tower Booking meets the Phase 1 criteria agreed with IT and sales: Palace records appear in the pipeline; a booking can be sent to the sandbox; a stay ending on day D frees the unit from D+2; a check-out creates a cleaning job; staff no longer re-type the same stay into Excel; and someone who is not an administrator cannot open administration. The instruments were a twelve-scenario script (S1–S12), a fifteen-line UI walk, written acceptance from each group, and the System Usability Scale completed after acceptance (Brooke, 1996). The environment was fixed as app-parktower-booking-dev with the Palace sandbox. A pass on a laptop alone, or a test stay written to live Palace or to app-parktower-booking-prod, would not have counted.

There is no automated test suite in the repository. Tests around clearance, overlaps and the Palace client were recommended in the handover rather than delivered during the placement, and daily use in production is not a substitute for the script. The 22 September clearance draft is a good example: it would have passed a quick happy-path check and still been wrong for sales, because a stay ending on 25 September must free the unit on the 27th, not the 28th. Following Forsgren et al. (2018), a failed scenario is recorded as a failure even when the Cursor history looks clean.

6.2. What was run

The twelve scenarios and the UI walk were run on the development App Service, and the results are reported in section 7.3: all twelve scenarios passed (Table 8), as did all fifteen UI lines. S7 deletes an unsent local row from the Edit modal. S9 passed because the application blocks an overlap when the booking is created; a conflict row only appears if the Palace dates later change. S11 is not a button but a confirmation from sales that the local booking was not re-entered in the spreadsheet. IT gave written acceptance on 22 October 2026 and Sales on 23 October 2026. Four System Usability Scale (SUS) responses were returned, three from Sales (mean 80.0) and one from IT (97.5), so both groups are above the planned threshold of 70 (Brooke, 1996). No further responses were collected.

Non-functional testing was limited. Easy Auth and AppUsers were exercised through normal sign-in and through S12, but there was no load test, no accessibility audit against the New Zealand Government Web Accessibility Standard, and no security review beyond keeping secrets out of Git (Pearce et al., 2022; Perry et al., 2023; New Zealand Government, 2025). Twelve passes do not make up for those gaps (Benedetti, 2026b).

6.3. What the script cannot show

A pass does not mean Cursor got the rule right on the first draft. The Vacant restore on 10 September and the clearance correction on 22 September both came from human review against a stated sales rule (section 7.2). The script also does not cover Admin → Delete booking (section 4.8), which arrived during hypercare after the workbook had been closed; it removes a local SENT_TO_PALACE row and does not void anything in Palace. Finally, the agreement on the RQ1 sample (Table 6) is a check on the labels, not a test of the product.

7. Results and discussion

This chapter reports what the development record can support. IT gave written acceptance on 22 October 2026 and Sales on 23 October 2026. The scripted scenarios, the UI checklist and the SUS means are in section 7.3, and the second coder's agreement on the twelve-item RQ1 sample is in Table 6. The RQ1 proportions below come from my own coding of the sixty-item corpus (Table 5).

7.1. RQ1 — task types and Cursor outcomes

The corpus is the sixty non-merge commits from 25 August to 13 October 2026 (Table 5), ending at 3ddff64; the last two rows (items 59 and 60, b41d2fb and 3ddff64) are both dated 13 October. Commit b424044 (22 October 2026; section 4.8) is outside the table. Item 2 (2cc393d) is the bulk import and is unclassified for Cursor outcomes. A mechanical sample of twelve items (starting at item 4 and taking every fifth) was drawn for the second coder on 13 October. After a further one-to-one check against the session transcripts on 20 October 2026, twenty-four of the sixty items are supported by a clear transcript showing Cursor producing or revising the change. The second coder's pack still contains transcripts only for the items the original sample required.

Task type was coded for all sixty items from the files changed. Assistant move is coded as Draft, Revise or Review only when a session transcript shows Cursor producing or changing the code for that commit. When no transcript clearly matches the commit, the move is recorded as None or unknown, which is a rule of the codebook and not a claim that Cursor was not used. Likewise, Accept, Amend, Reject and attributable rework are only treated as Cursor outcomes where there is a transcript; elsewhere those columns are set to no. Table 5

holds the codes for all sixty items. The RQ1 proportions in Section 7 use the twenty-four identifiable items as the denominator.

Cursor's involvement could be identified for 24 of the 60 non-merge commits (40%). The other 36 remain None or unknown because no transcript clearly shows Cursor producing or revising that commit's diff. A chat on the same day was not considered enough, the bulk import 2cc393d was not forced into a category, and the five IT workflow commits show no sign of Cursor. Among the 24 identifiable items (Table 5), 18 were accepted (75%) and 6 were amended (25%). Reject and rework are separate flags, not a third share of that split: the clearance off-by-one (item 49) is one of the six amendments and is also the one rejected draft (4%), and 5 items later needed attributable rework (21%). Six items were identifiable in the first coding pass, and eighteen more were added on 20 October 2026 after a one-to-one check against the session transcripts, including four that already had excerpts in the 13 October second-coder sample. Commits where the match was doubtful were not added. Second-coder agreement is reported only for the twelve-item sample drawn on 13 October (Table 6) and was not extended to these eighteen items.

Table 5 - RQ1 coding of all sixty non-merge commits in the development window.

Item	Date	Hash	Author	Subject	Task type	Assistant move	Acc	Amd	Rej	Rew	Unc	Tx
1	2026-08-25	c90522e	andrecrockers	Initial commit	Unclassified	None or unknown	no	no	no	no	yes	no
2	2026-08-25	2cc393d	andrecrockers	Changing Rep	Unclassified	None or unknown	no	no	no	no	yes	no
3	2026-08-25	ba6dbfc	andrecrockers	Update .gitignore	Configuration and deployment	None or unknown	no	no	no	no	no	no
4	2026-08-25	575e59b	andrecrockers	Delete english-only.mdc	Documentation	None or unknown	no	no	no	no	no	no
5	2026-08-25	8eedeb5	andrecrockers	Update .gitignore	Configuration and deployment	None or unknown	no	no	no	no	no	no
6	2026-08-25	e0ba3b8	andrecrockers	.	Rule logic	None or unknown	no	no	no	no	no	no
7	2026-08-27	7e2d1c2	hamida-crockers	Add gunicorn dependency to requirements.tx	Configuration and deployment	None or unknown	no	no	no	no	no	no
8	2026-08-27	9a43a36	andrecrockers	Review 270826	UI	None or unknown	no	no	no	no	no	no
9	2026-08-27	9034b98	andrecrockers	Update config.py	Configuration and deployment	Revise	yes	no	no	no	no	yes
10	2026-08-27	9427315	hamida-crockers	Add or update the Azure App Service build	Configuration and deployment	None or unknown	no	no	no	no	no	no
11	2026-09-01	33b41f2	andrecrockers	change auth variables	Configuration and deployment	None or unknown	no	no	no	no	no	no
12	2026-09-01	c10368f	andrecrockers	API varibles changes and Update on Confli	Integration	Revise	no	yes	no	no	no	yes
13	2026-09-01	a69fbb7	andrecrockers	update on the conflicts	Rule logic	Revise	no	ye	no	ye	no	ye

Item	Date	Hash	Author	Subject	Task type	Assistant move	Acc	Amd	Rej	Rew	Unc	Tx
				and conflicts page				s		s		s
14	2026-09-01	d7adfc8	andrecrockers	Easy Auth	Configuration and deployment	Draft	yes	no	no	no	no	yes
15	2026-09-08	cafd1f0	andrecrockers	Harden Easy Auth AppUsers lookup for Azure	Configuration and deployment	Revise	yes	no	no	no	no	yes
16	2026-09-08	d65e04a	andrecrockers	Cleaning Schedule	Rule logic	None or unknown	no	no	no	no	no	no
17	2026-09-08	5124acd	andrecrockers	Conflicts	Rule logic	Revise	yes	no	no	no	no	yes
18	2026-09-08	485f690	andrecrockers	Variable Change	Integration	None or unknown	no	no	no	no	no	no
19	2026-09-08	ccf2910	andrecrockers	Find Avalible Unit	UI	Revise	yes	no	no	no	no	yes
20	2026-09-08	35a44e1	andrecrockers	Find Avalible Unit	UI	None or unknown	no	no	no	no	no	no
21	2026-09-08	0dd6801	andrecrockers	Tenancy Dear + Clean Schedule UI	UI	None or unknown	no	no	no	no	no	no
22	2026-09-08	c2b3e21	andrecrockers	Cleaning Schedule	UI	Revise	no	yes	no	no	no	yes
23	2026-09-08	d484b3c	andrecrockers	Cleaning Schedule	UI	None or unknown	no	no	no	no	no	no
24	2026-09-08	6bd03c2	andrecrockers	Update to not include unecessary files at	Configuration and deployment	None or unknown	no	no	no	no	no	no
25	2026-09-08	66a787f	andrecrockers	Data to Palace, Find Available Units and P	Integration	None or unknown	no	no	no	no	no	no
26	2026-09-10	487de75	andrecrockers	Rule to show Vacant	Rule logic	Revise	yes	no	no	yes	no	yes
27	2026-09-10	c190cbf	andrecrockers	Update Readme.md	Documentation	Revise	yes	no	no	no	no	yes
28	2026-09-10	e084fa3	andrecrockers	Change on Edit button	UI	None or unknown	no	no	no	no	no	no
29	2026-09-10	45f64f9	andrecrockers	Conflicst	Rule logic	None or unknown	no	no	no	no	no	no
30	2026-09-10	a5c2478	andrecrockers	Change Check-in date	Rule logic	None or unknown	no	no	no	no	no	no
31	2026-09-10	686286b	andrecrockers	adding dismiss button	UI	Revise	yes	no	no	no	no	yes
32	2026-09-14	09b7117	hamida-crockers	Add or update the Azure App Service build	Configuration and deployment	None or unknown	no	no	no	no	no	no
33	2026-09-15	7c31699	hamida-crockers	Remove the Azure App Service build and dep	Configuration and deployment	None or unknown	no	no	no	no	no	no
34	2026-09-15	5b0e932	hamida-crockers	Add or update the Azure App Service build	Configuration and deployment	None or unknown	no	no	no	no	no	no
35	2026-09-15	495f20f	andrecrockers	Local Scrip and SLN update	Unclassified	None or unknown	no	no	no	no	yes	no
36	2026-09-15	0095547	andrecrockers	New Conflict	UI	Draft	yes	no	no	no	no	yes
37	2026-09-15	d6d7c52	andrecrockers	.	UI	None or unknown	no	no	no	no	no	no
38	2026-09-17	45e2dcb	andrecrockers	Adjust on available rule	Rule logic	Revise	yes	no	no	no	no	yes
39	2026-09-17	ac91cbf	andrecrockers	Correction at Check-out Calendar	UI	Revise	no	yes	no	no	no	yes
40	2026-09-17	1e81ebe	andrecrockers	Changes Requested by James and Laura	UI	None or unknown	no	no	no	no	no	no
41	2026-09-17	55641ea	andrecrockers	update values for Unit features	Integration	None or unknown	no	no	no	no	no	no
42	2026-09-17	896fc74	andrecrockers	DB update	UI	None or unknown	no	no	no	no	no	no
43	2026-09-22	7f6fd6f	andrecrockers	Update db_schema.py	Integration	None or unknown	no	no	no	no	no	no
44	2026-09-22	fdbf105	andrecrockers	Add new Maintenance doc + update do urllib	Documentation	None or unknown	no	no	no	no	no	no
45	2026-09-22	a218586	andrecrockers	Guide Update	Documentation	None or unknown	no	no	no	no	no	no
46	2026-09-22	bc4e717	andrecrockers	Delete ~$rk Tower App Maintenance_Guide.do	Documentation	None or unknown	no	no	no	no	no	no
47	2026-09-22	38044dc	andrecrockers	Components update	Documentation	None or unknown	no	no	no	no	no	no
48	2026-09-22	b5ca207	andrecrockers	.	Documentation	None or unknown	no	no	no	no	no	no
49	2026-09-22	235e93b	andrecrockers	Adjust on the available unit rule	Rule logic	Revise	no	yes	yes	yes	no	yes
50	2026-09-22	8ed1b98	andrecrockers	Conflicts	Rule logic	None or unknown	no	no	no	no	no	no
51	2026-09-29	66dfc15	andrecrockers	Missing Unit 106 and Add Block to unit fea	Rule logic	Revise	yes	no	no	no	no	yes
52	2026-09-29	b54c977	andrecrockers	Button text change	UI	Revise	ye	no	no	no	no	ye

Item	Date	Hash	Author	Subject	Task type	Assistant move	Acc	Amd	Rej	Rew	Unc	Tx
							s					s
53	2026-10-08	001543c	andrecrockers	Performance Review	UI	Draft	yes	no	no	no	no	yes
54	2026-10-08	e8a3aec	andrecrockers	Correction on Auth	Configuration and deployment	Revise	yes	no	no	yes	no	yes
55	2026-10-08	213b59d	andrecrockers	.	Configuration and deployment	None or unknown	no	no	no	no	no	no
56	2026-10-08	155ecc4	andrecrockers	Change on Conflicts	UI	Revise	yes	no	no	no	no	yes
57	2026-10-08	5879f80	andrecrockers	Performance Update	UI	Revise	yes	no	no	no	no	yes
58	2026-10-08	305b47e	andrecrockers	Performance review	UI	Revise	yes	no	no	yes	no	yes
59	2026-10-13	b41d2fb	andrecrockers	Booking without room	UI	Draft	yes	no	no	no	no	yes
60	2026-10-13	3ddff64	andrecrockers	Docs Update	Documentation	Revise	no	yes	no	no	no	yes

Note: Acc = Accept; Amd = Amend; Rej = Reject; Rew = attributable rework; Unc = Unclassified; Tx = clear transcript (yes/no). Table 5 lists the sixty non-merge commits coded for RQ1, from 25 August through 13 October 2026. Commit b424044 (22 October 2026; section 4.8) falls inside the placement calendar but is not a sixty-first row: there was not time to recode the corpus, redraw the second-coder sample, or repeat the validation workbook with IT. Item 2 is the bulk import and is Unclassified. Twenty-four rows have Tx = yes (40%); thirty-six show Assistant move as None or unknown. Item 59 is the Week 12 Booking / Tenant Booking change (b41d2fb). When a commit hash from that corpus is named elsewhere in this report, the corresponding row is in Table 5. The exception is b424044 (section 4.8), which is listed and not coded. Second-coder agreement uses only the twelve-item sample drawn on 13 October (Table 6); the eighteen later transcript promotions are not mixed into that concordance.

The second coder returned the worksheet for the 13 October sample without having seen my labels. Task type was compared across all twelve items. Assistant move, Accept, Amend, Reject and attributable rework were compared only on the six items that had a transcript in the pack (items 9, 14, 19, 39, 49 and 59). Item 54 was later matched to a transcript in my sixty-item coding (Table 5), but that change is not part of this comparison. For the six items without a transcript, we both coded the assistant move as None or unknown, which tests the codebook rule rather than Cursor's work.

Table 6 - Second-coder agreement on the 13 October RQ1 sample

Column	n	Agree	Percent	Items that differed
Task type	12	7	58%	4, 14, 29, 39, 54
Assistant move	6	5	83%	59 (Draft / Revise)
Accept	6	5	83%	39
Amend	6	4	67%	39, 49
Reject	6	5	83%	49
Attributable rework	6	5	83%	39

Note: Transcript rows only for Cursor outcomes. n = 12 for Task type; n = 6 for the other columns (send-pack transcripts). Cohen's κ is 0.47 for Task type and 0.57 for Assistant move. Those values are a check on the percentages, not a reliability claim for the sixty-item corpus.

Task type was the weaker column, with agreement on 7 of 12 items (Cohen's $\kappa = 0.47$). The same diff can reasonably be read as configuration, integration, a screen change or

a defect fix. Item 14 (Easy Auth) was Configuration and deployment for me and Integration for her, item 39 (calendar.js) was UI against Defect fix, and item 29 was Rule logic against Integration. Agreement on Cursor outcomes was closer. We agreed on the assistant move for five of the six transcript items ($\kappa = 0.57$), the exception being item 59, which I coded as Draft and she coded as Revise. Accept, Reject and rework each agreed on five of six, and Amend on four of six. The two difficult items were 39 and the 22 September clearance change (item 49). On item 49 we both coded Rule logic and rework, but she marked it Unclassified rather than Amend or Reject, whereas I had coded both Amend and Reject against the sales date rule. The sixty-item proportions in Table 5 remain my coding; Table 6 shows where another reader, working from the same artefacts, reached a different label.

7.2. RQ2 — human control points

RQ2 asks which human control points were kept during development, and what evidence shows they kept ownership of the product rules and of correctness (Benedetti, 2026b). A control point only counts if it is backed by a commit, a transcript excerpt or a documented test incident, and a type is treated as recurring when it appears in three or more work items (or three clearly separate artefacts). Daily use of the application is not treated as sign-off. Written acceptance is on file from IT (22 October 2026) and from Sales (23 October 2026), which gives two artefacts, so this type is not recurring.

Table 7 - Human control points retained during LLM-assisted development.

Control type	Recurring?	Evidence in this project
Rule stated first	Yes	Items 49, 51, 59; also 4 Aug exit-date rule, 11–13 Aug listing/Vacant rules, and the interim +1-day clearance instruction on 22 Sep (human, not a Cursor error).
Review against rule	Yes	Items 13, 26, 49; Pipeline date check on 25 Aug; auto-search kept as a requirement on 8 Oct.
Server-side placement	At the limit	Three file placements: conflict_service (item 13),

		Occupancy.pick_display (item 38), clears_before_entry (item 49). Evidence is where the rule runs, not a verbal “put it on the server”.
Routed through IT	Yes	Items 7, 10, 32, 33, 34 — five hamida-crockers commits on gunicorn and Azure workflows.
Secret refused	No	None. A .gitignore change that listed folders was not a refused credential.
Stakeholder sign-off	No	Written IT acceptance: Hamida Khan, 22 October 2026 (Sign_off IT.pdf). Written Sales acceptance: 23 October 2026. Confirmed Palace load and send, scenarios without unresolved conflicts, check-out cleaning records, replacement of Excel tracking, and roomless Tenant Booking. Daily use is not counted. Two artefacts are not recurring.

The answer to RQ2 is that five kinds of human control are supported by artefacts and one is not. Rule stated first and Review against rule both recurred: product rules were written down before Cursor coded them, and drafts were checked against those rules, including the Vacant restore on 10 September and the clearance off-by-one on 22 September. Routed through IT also recurred, through five commits by an IT engineer covering gunicorn and the Azure deployment workflows. Server-side placement appeared three times, in the files where the availability and conflict rules run (occupancy.py and conflict_service.py); that is enough to list but too thin to call a habit. Stakeholder sign-off is supported by two written acceptances. IT accepted the Phase 1 points on 22 October 2026, covering the Palace load and send, scenarios without unresolved conflicts, cleaning records from check-outs, the replacement of the Excel tracking and the roomless Tenant Booking path, and Sales accepted on 23 October 2026. Secret refused did not appear at all.

Two episodes show the pattern most clearly (section 4.5). On 10 September (item 26, commit 487de75), a Vacant display rule based on a seven-day gap had disappeared, and I told

the assistant to restore it and not to drop rules unless asked. On 22 September (item 49, commit 235e93b), after I asked for a two-day clearance after check-out, Cursor implemented a check that would have freed a stay ending on 25 September only on 28 September. Under the sales rule, first entry is check-out plus two, so the unit is free on 27 September. I rejected the off-by-one against that rule, and the correction sits on the server in occupancy.py (clears_before_entry), not only in the browser.

Routed through IT is the strongest recurring type in the Git record, because it reflects platform rights I do not hold. Secret refused remains empty, and stakeholder sign-off has two artefacts, one from IT (22 October 2026) and one from Sales (23 October 2026), which is not enough to count as recurring. Two things were deliberately not counted as Rule stated first: writing comments only in English, which is a coding convention, and the 17 September plan titled “schema before code”, which was not a statement of a product rule.

7.3. Product validation

Scripted scenarios S1–S12 and the fifteen-line UI checklist were run on app-parktower-booking-dev against the Palace sandbox, not on a laptop alone, and no test stays were written to live Palace or to app-parktower-booking-prod. All twelve scenarios passed (Table 8), as did all fifteen UI lines; a failed scenario would have been recorded as a failure regardless of how the Cursor history looked. IT’s acceptance is dated 22 October 2026 and Sales’ acceptance 23 October 2026. Four System Usability Scale (SUS) responses were returned, three from Sales (mean 80.0) and one from IT (97.5), and both group means are above the planned threshold of 70 (Brooke, 1996). No further responses were collected.

The free-text comments from Sales add context to those scores. One comment described the effect on the desk directly: “It’s absolutely helping us immensely with our bookings.” Another pointed to something the application does not yet do: “It would be great to be able to

filter out a specific level and see who currently has moved in and what rooms are available or already booked." That request is for a view of current occupancy by floor, which the Phase 1 screens do not provide, and it is carried into future work in section 8.3. Given how few people responded, the comments are reported without linking them to individual scores.

Table 8 - Scripted product scenarios on Dev against the Palace sandbox.

ID	Title	Result	Note
S1	Palace load	Pass	Sandbox tenancies on Pipeline
S2	Clearance D+1	Pass	Unit not offered
S3	Clearance D+2	Pass	Unit offered if nothing else blocks
S4	Local pipeline booking	Pass	Add to pipeline or Tenant Booking; not sent
S5	Send to Palace sandbox	Pass	Not live Palace
S6	After send — no edit/resend	Pass	Edit and Send disabled
S7	Delete local unsent row	Pass	Edit modal only; not Admin delete
S8	Check-out and cleaning	Pass	Warning clears after schedule
S9	Overlap and clearance	Pass	App blocks create; Palace drift shows in Conflicts
S10	Export New	Pass	New jobs only
S11	No double entry	Pass	Sales confirmation, not a click
S12	Non-admin access	Pass	Admin routes refused

S7 covers the Edit → Delete path for an unsent local row; Admin → Delete booking (section 4.8) is outside the script. S9 passed because the application blocks an overlap when the booking is created. The workbook notes that a conflict row only appears when Palace dates later diverge, not when someone at the desk tries to book the clash in the app. S11 is a confirmation from Sales rather than a click: the local booking created in S4 was not re-typed into the spreadsheet.

8. Conclusion and future work

Park Tower Booking replaced double entry for one Crockers building, and sales now work from a single pipeline connected to Palace. On the development App Service, the Phase 1 script passed all twelve scenarios (Table 8) and all fifteen UI lines. IT accepted these points in writing on 22 October 2026 and Sales on 23 October 2026. The System Usability Scale means were 80.0 for Sales (n = 3) and 97.5 for IT (n = 1), both above the planned threshold of 70 (Brooke, 1996).

Cursor’s involvement can be seen in a commit and a matching transcript for 24 of the 60 non-merge items (40%). Of those, 18 were accepted (75%) and 6 were amended (25%). The 22 September clearance off-by-one is one of the six amendments and the one rejected draft; 5 items later needed attributable rework (21%). On the twelve-item sample, the second coder agreed with me more often on Cursor outcomes than on task type (Table 6). These percentages are my coding of a closed corpus, and they say nothing about whether the assistant made the work faster.

Where the record can show it, people kept the booking rules. They were written down before Cursor coded them, drafts were checked against those rules, platform work went through IT, and availability and conflicts run on the server (Table 7). Refusing a secret never came up. Both IT and Sales signed off in writing.

8.1. Approximate development effort (LOC-based comparison)

To put the Cursor-assisted build in context without inventing a timesheet, I estimated development and integration effort from the size of the repository rather than from remembered hours. The starting point was roughly 18,700 lines of productive code across Python, JavaScript and HTML (about 11,200, 5,700 and 1,800 lines respectively), SUS,

second-coder work, sign-off correspondence and other course-only tasks. Hours were calculated by dividing those lines by an explicitly assumed net productivity (lines implemented, reviewed and integrated per hour) under three scenarios: with AI assistance (Cursor), an experienced developer without AI, and a junior developer without AI. The sixty non-merge commits and the 25 August to 31 October 2026 window were treated as limits on the activity, not as hours worked.

The same size and scope brief was given to several large language models (Gemini, DeepSeek, ChatGPT and Claude) and to Cursor, so that differences in the results would come from different productivity assumptions rather than different codebases. All of them treated lines of code as an imperfect measure of size, and all warned that published AI speed-ups (for example Peng et al. (2023) on a small specified task; Becker et al. (2025) on experienced open-source work; and Moradi Dakhel et al. (2023) on the effect of experience) cannot simply be applied to this repository as a multiplier. DeepSeek declined to give a three-way split. Instead it offered a single, cautious range for AI-assisted work, assuming that only 70% to 90% of the code was development or integration and using a lower productivity band.

Table 9 - LOC-based development/integration effort ranges from five AI sources

Source	With AI (h)	Experienced, no AI (h)	Junior, no AI (h)
Cursor (Composer)	290–420	470–750	850–1,550
Gemini	220–300	620–935	1,350–2,300
ChatGPT	267–468	534–935	1,247–2,338
Claude	190–375	750–1,250	1,550–2,350
DeepSeek	430–1,120*	—	—

**DeepSeek reported one AI-assisted reference range (70%–90% of LOC at 15–30 LOC/h) and did not publish separate experienced/junior splits.*

Taken together, most of the AI-assisted ranges fall between about 190 and 470 hours, with DeepSeek's more conservative range higher at 430 to 1,120 hours. Estimates for an experienced developer without AI range from roughly 470 to 1,250 hours, and for a junior developer without AI from roughly 850 to 2,350 hours. These spreads reflect the productivity assumptions each model chose, not a measured speed-up from Cursor on Park Tower

Booking. Lines of code also leave out the trial and error with Palace, authentication and Azure configuration, the churn from rewritten drafts, and the manual verification needed in the absence of automated tests. The table should therefore be read as an indicative comparison of scenarios for development and integration only. It is not a reconstruction of the hours actually spent, and it does not show that AI made delivery faster by any fixed percentage.

8.2. Limitations

This is a single case: one building, one developer and one assistant. I was both the developer of Park Tower Booking and the researcher who coded its record, which limits the independence of the analysis. Usability was rated by only four people, one from IT and three from sales. The requests I made to the assistant should also have been logged more carefully at the time; because most commits were matched to a session weeks later, Cursor's involvement could only be identified for 24 of the 60 items.

Park Tower Booking is a brownfield business system whose rules belong to a working sales desk, and it was built to solve a business problem rather than set up as a research experiment, so a randomised comparison of AI and non-AI work was never an option (Becker et al., 2025; Runeson & Höst, 2009). RQ1 cannot see edits that left no transcript, and Table 5 stops at 13 October 2026, so commit b424044 is described in section 4.8 but not coded. The second coder's check on Cursor outcomes rests on only six transcript items. Without an automated test suite, verification relied on the script, the UI walk and review against the sales rules.

The effort ranges in Table 9 apply assumed productivity rates to the size of the repository rather than coming from a timesheet, and published Copilot speed-ups cannot be transferred to this system as a multiplier (Peng et al., 2023; Moradi Dakhel et al., 2023). Finally, Section

5 sets out the Privacy Act duties and the still-unwritten purpose for sending school, country and gender to Palace, which remain limitations of the system left in use.

8.3. Future work

For the application, the handover recommendations still stand: automated checks on clearance and missing cleans, a written purpose and retention rule for school, country and gender, and an accessibility audit. Sales also asked for a view by building level showing who has moved in and which rooms are available or already booked (section 7.3), which would be a natural next screen for the desk. For research, the main lesson concerns the record itself. Because most commits were matched to sessions weeks after the work, Cursor's involvement could only be identified for 24 of the 60 items. A future study should log the development phase as it happens, linking each Cursor session to its commit at the time instead of reconstructing the link from chat history. A suggested next study would run two arms on the same protocol, one with an LLM and one without, with developers at more than one experience level and with activities split by difficulty: from a fully specified task through to a tacit brownfield rule. Hours would be recorded on a timesheet so the arms can be compared. Automatic checks on clearance and cleaning would catch the 22 September class of error without waiting for a sales review. Those outcomes can then be compared across people, rather than resting on a single case. If it aims to say anything about effort, it should record hours instead of dividing repository size by assumed rates. The most useful next case would be another brownfield system whose rules belong to the people using it, rather than another small, fully specified coding task. The 22 September off-by-one is a reminder of why: code can read fluently and still get the sales date wrong.

References

Association for Computing Machinery. (2018). ACM code of ethics and professional conduct. Retrieved October 1, 2026, from https://www.acm.org/code-of-ethics

Barke, S., James, M., & Polikarpova, N. (2023). Grounded Copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages, 7*(OOPSLA1), 85–111. https://doi.org/10.1145/3586030

Becker, J., Rush, N., Barnes, B., & Rein, D. (2025). *Measuring the impact of early-2025 AI on experienced open-source developer productivity.* Model Evaluation & Threat Research (METR). https://doi.org/10.48550/arXiv.2507.09089

Benedetti, A. S. (2026, September 5). The Usage of LLMs in Real Software Development: A Use Case from Practice - Assessment 1 - Research Development and Plan. Auckland.

Benedetti, A. S. (2026, October 26). The Usage of LLMs in Real Software Development: A Use Case from Practice - Assessment 2 - Work in Progress Report. Auckland: Yoobee College of Creative Innovation.

Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology, 3*(2), 77-101. https://doi.org/10.1191/1478088706qp063oa

Brooke, J. (1996). SUS: A "quick and dirty" usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), *Usability Evaluation in Industry* (pp. 189–194). Taylor & Francis. https://doi.org/10.1201/9781498710411-35

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P., & Kaplan, J. (2021). *Evaluating Large Language Models Trained on Code.* San Francisco: arXiv. https://doi.org/10.48550/arXiv.2107.03374

Chen, X., Lin, M., Schärli, N., & Zhou, D. (2024). Teaching Large Language Models to Self-Debug. *The Twelfth International Conference on Learning Representations.* Vienna: OpenReview. https://doi.org/10.48550/arXiv.2304.05128

De Brito, V. M., & Farias, K. (2025). *Understanding the Role of Large Language Models in Software Engineering: Evidence from an Industry Survey.* Sao Leopoldo: arXiv. https://doi.org/10.48550/arXiv.2512.21347

Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps: Building and scaling high performing technology organizations.* IT Revolution Press.

Gomes, V., Selb, D., Palomba, F., Spinola, R., & Lo, D. (2026). *LLM-assisted empirical software engineering: Systematic literature review and research agenda.* arXiv. https://doi.org/10.48550/arXiv.2604.26192

Hevner, A., March, S., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75-105. https://doi.org/10.2307/25148625

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., & Schmidhuber, J. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. *The Twelfth International Conference on Learning Representations.* Vienna: OpenReview. https://doi.org/10.48550/arXiv.2308.00352

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology, 33*(8), 1-79. https://doi.org/10.1145/3695988

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation.* Addison-Wesley.

Jiang, N., Lutellier, T., & Tan, L. (2021). CURE: Code-Aware Neural Machine Translation for Automatic Program Repair. *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)* (pp. 1161-1173). Madrid: IEEE. https://doi.org/10.1109/ICSE43902.2021.00107

Jimenez, C., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? *The Twelfth International Conference on Learning Representations.* https://doi.org/10.48550/arXiv.2310.06770

Kitchenham, B., & Charters, S. (2007). *Guidelines for performing Systematic Literature Reviews in Software Engineering (Version 2.3).* Keele: Keele University and University of Durham. Retrieved 08 09, 2026

Konstantinou, M., Degiovanni, R., & Papadakis, M. (2026). How well LLM-based test generation techniques perform with newer LLM versions? *2026 IEEE International Conference on Software Testing, Verification and Validation (ICST)* (pp. 233-243). IEEE. https://doi.org/10.1109/ICST69053.2026.00040

Kukutai, T., & Taylor, J. (2016). *Indigenous data sovereignty: Toward an agenda.* ANU Press. https://doi.org/10.22459/CAEPR38.11.2016

Liang, J., Yang, C., & Myers, B. (2024). A large-scale survey on the usability of AI programming assistants: Successes and challenges. *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Article 52).* Association for Computing Machinery. https://doi.org/10.1145/3597503.3608128

Moradi Dakhel, A., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M., & Jiang, Z. (2023). GitHub Copilot AI pair programmer: Asset or liability? *Journal of Systems and Software, 203*, 111734. https://doi.org/10.1016/j.jss.2023.111734

Mozannar, H., Bansal, G., Fourney, A., & Horvitz, E. (2024). Reading between the lines: Modeling user behavior and costs in AI-assisted programming. *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (Article 142).* Association for Computing Machinery. https://doi.org/10.1145/3613904.3641936

New Zealand Government. (2025). *Web Accessibility Standard 1.2.* Department of Internal Affairs. Retrieved October 1, 2026, from https://www.digital.govt.nz/standards-and-guidance/nz-government-web-standards/web-accessibility-standard-1-2/about-the-web-accessibility-standard

New Zealand Parliament. (1993). Human Rights Act 1993. Retrieved October 1, 2026, from https://www.legislation.govt.nz/act/public/1993/0082/latest/DLM304212.html

New Zealand Parliament. (2020). Privacy Act 2020. Retrieved October 1, 2026, from https://www.legislation.govt.nz/act/public/2020/0031/latest/LMS23223.html

Office of the Privacy Commissioner. (2022). *Data privacy in the rental sector.* Retrieved October 1, 2026, from https://www.privacy.org.nz/assets/New-order/Resources-/Publications/Insights-reports/Consumer-and-OPC-Report-Data-privacy-in-the-rental-sector-September-2022.pdf

Page, M., McKenzie, J., Bossuyt, P., Boutron, I., Hoffmann, T., Mulrow, C., Shamseer, L., Tetzlaff, J., Akl, E., Brennan, S., Chou, R., Glanville, J., Grimshaw, J., Hróbjartsson, A., Lalu, M., Li, T., Loder, E., Mayo-Wilson, E., McDonald, S., McGuinness, L., . . . Moher, D. (2021). The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ, 372*, n71. https://doi.org/10.1136/bmj.n71

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. *2022 IEEE Symposium on Security and Privacy (SP)* (pp. 754-768). IEEE. https://doi.org/10.1109/SP46214.2022.9833571

Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv*. https://doi.org/10.48550/arXiv.2302.06590

Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). Do users write more insecure code with AI assistants? *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (pp. 2785–2799). Association for Computing Machinery. https://doi.org/10.1145/3576915.3623157

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., & Sun, M. (2024). ChatDev: Communicative Agents for Software Development. In L.-W. Ku, A. Martins, & V. Srikumar (Ed.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 15174–15186). Bangkok: Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.acl-long.810

Rao, H., Zhao, Y., Hou, X., Wang, S., & Wang, H. (2025). *Software Engineering for Large Language Models: Research Status, Challenges and the Road Ahead.* Wuhan: arXiv. https://doi.org/10.48550/arXiv.2506.23762

Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering, 14*(2), 131-164. https://doi.org/10.1007/s10664-008-9102-8

Schäfer, M., Nadi, S., Eghbali, A., & Tip, F. (2024). An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering, 50*(1), 85-105. https://doi.org/10.1109/TSE.2023.3334955

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Te Mana Raraunga. (2018). Principles of Māori data sovereignty. Māori Data Sovereignty Network. Retrieved October 1, 2026, from https://static1.squarespace.com/static/58e9b10f9de4bb8d1fb5ebbc/t/5bda208b4ae237cd89ee16e9/1541021836126/TMR+Ma%CC%84ori+Data+Sovereignty+Principles+Oct+2018.pdf

Trofimov, A. (2025, March 12). *Enhancing Software Development with AI Tools: A Practical Look at Cursor IDE*. Retrieved 08 09, 2026, from ELEKS. https://eleks.com/research/cursor-ide/

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2.* Retrieved October 1, 2026, from https://www.w3.org/TR/WCAG22/

Ziegler, A., Kalliamvakou, E., Li, X., Rice, A., Rifkin, D., Simister, S., Sittampalam, G., & Aftandilian, E. (2024). Measuring GitHub Copilot's impact on productivity. *Communications of the ACM, 67*(3), 54–63. https://doi.org/10.1145/3633453